

Mathematics of Model-Based Systems

John Morris

2 March 2026

Presentation to University of Alabama-Huntsville

Biographical Sketch



BS: Mech Engr, BYU (2021)
Minor in Computer Science



PLM Applications Engineer (2021-2025)



MS: Mech Engr, Clemson (2023)
Emphases in manufacturing, design, and dynamics



Collaboration with NPS (2024)



Visiting Researcher at NIST (2025)



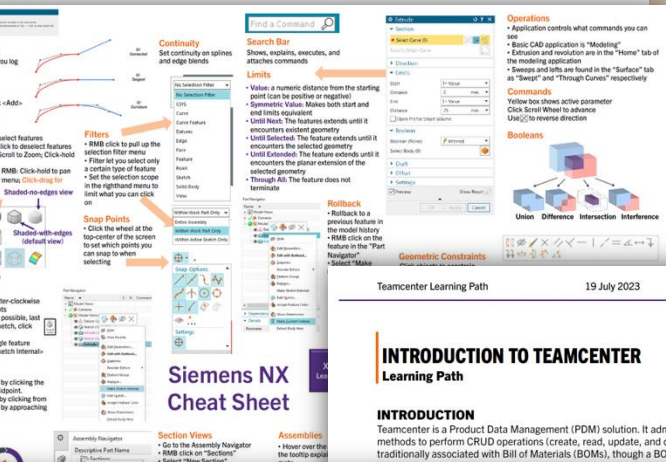
PhD: Mech Engr, Clemson (2025)



Collaboration with Alan Turing Institute (2026)



PLM Applications Engineer



Siemens NX Cheat Sheet

Continuity: Set continuity on splines and edge blends.

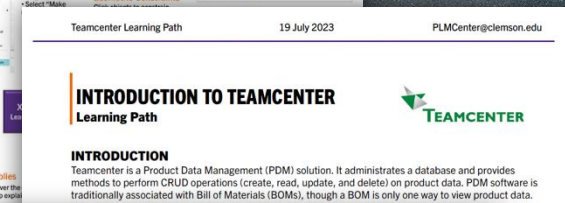
Search Bar: Shows, explores, executes, and attaches commands.

Filters: Filter to select only a certain type of feature.

Shade: Click-hold to pan menu. Click-hold to zoom. Click-hold to select.

Shaded-as-edges (default view): Click the wheel at the top-center of the screen to select when selecting.

Section Views: Go to the Assembly Navigator. RMB click on "Section". Select "New Section". Remove section view. RMB on the section view. RMB on the section view.



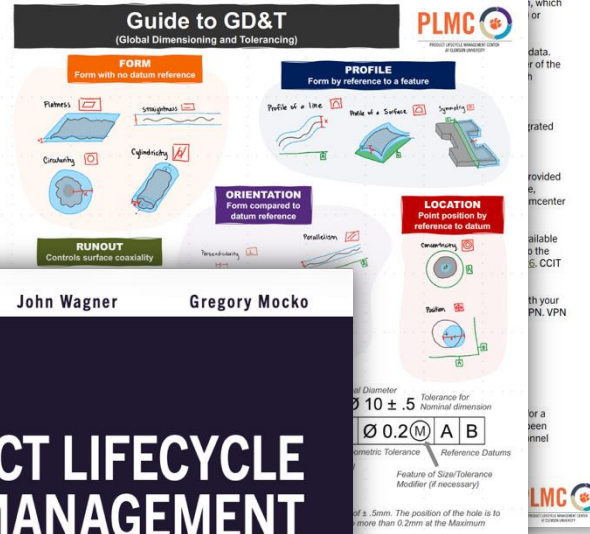
Teamcenter Learning Path

19 July 2023 PLMCenter@clemsun.edu

INTRODUCTION TO TEAMCENTER Learning Path

INTRODUCTION

Teamcenter is a Product Data Management (PDM) solution. It administrates a database and provides methods to perform CRUD operations (create, read, update, and delete) on product data. PDM software is traditionally associated with Bill of Materials (BOMs), though a BOM is only one way to view product data.



Guide to GD&T
(Global Dimensioning and Tolerancing)

FORM
Form with no datum reference

PROFILE
Form by reference to a feature

ORIENTATION
Form compared to datum reference

LOCATION
Point position by reference to datum

RUNOUT
Controls surface coaxiality

Geometric Symbols Table:

Name	Icon	Description
Flatness		Surface deviation from plane
Straightness		Edge deviation from line
Circularity		Shift deviation from circle
Cylindricity		Shift deviation from cylinder
Perpendicularity		Feature deviation from perpendicular
Angularity		Orientation deviation from datum
Parallelism		Feature deviation from offset line
Profile (Line)		Line deviation from nominal
Profile (Surface)		Surface deviation from nominal
Symmetry		Feature deviation across plane
Concentricity		Point deviation from circle center

Product Lifecycle Management
Processes for a Modern Industry

John Morris John Wagner Gregory Mocko



4.2 Product Data Management

PREV **NEXT**

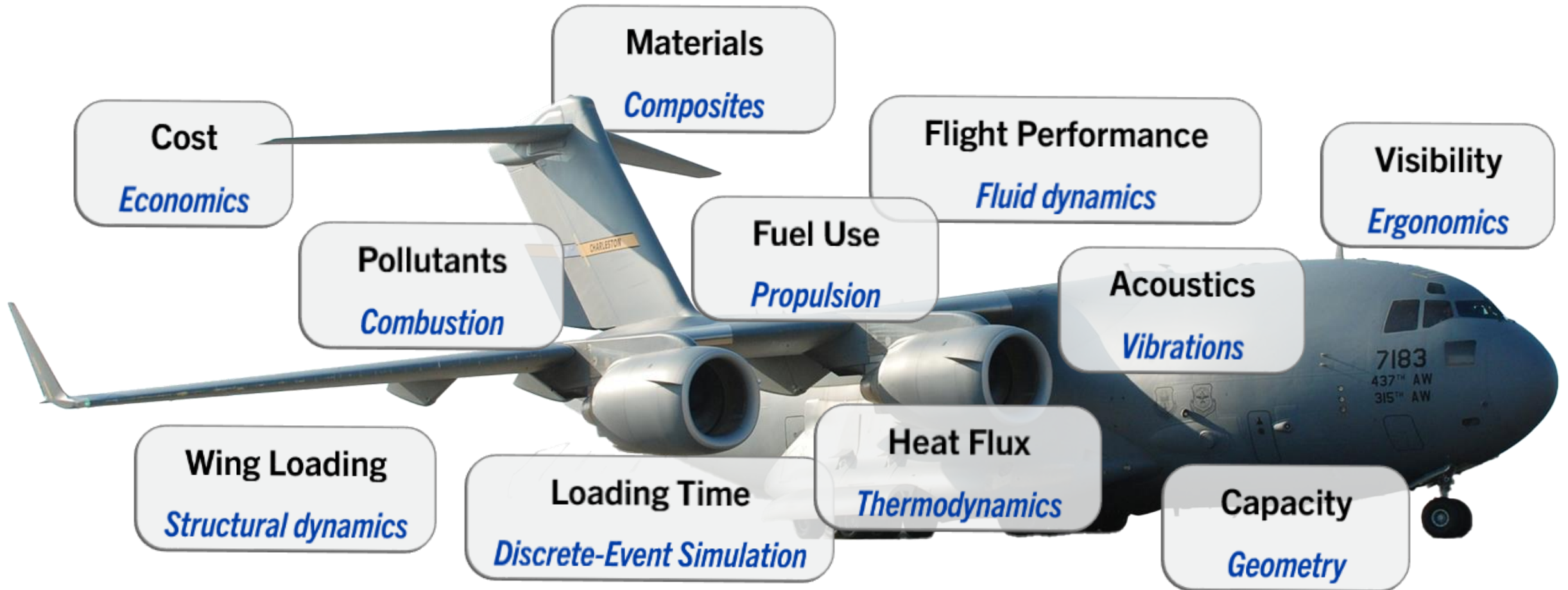
The basics of product data management (PDM) and repository development are covered in this course. The topics include product structure, configuration management, requirements modeling, SysML, analysis data management, interoperability, and data exchange.



Science is the work of describing the worlds around us

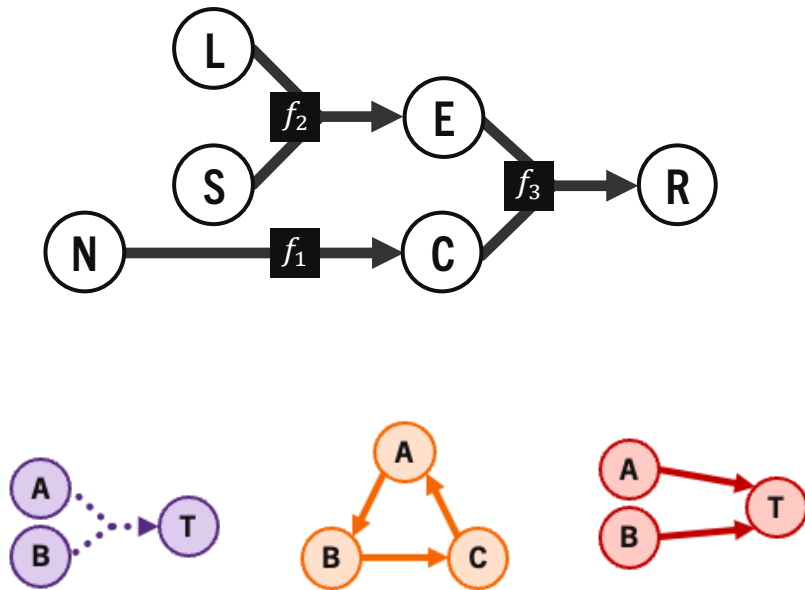


Knowing information about complex systems require models across many domains

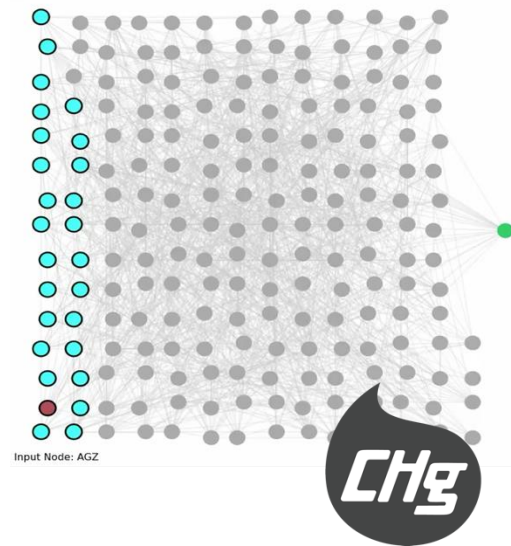


Constraint hypergraphs are a framework for universal, declarative systems modeling

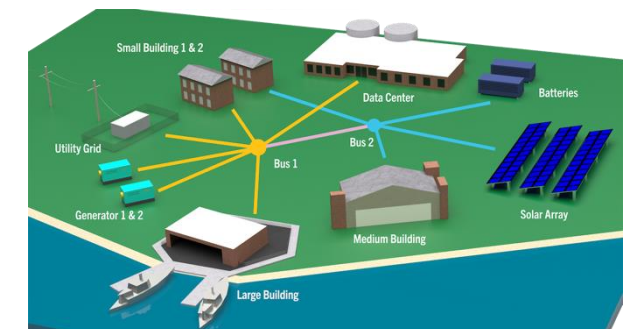
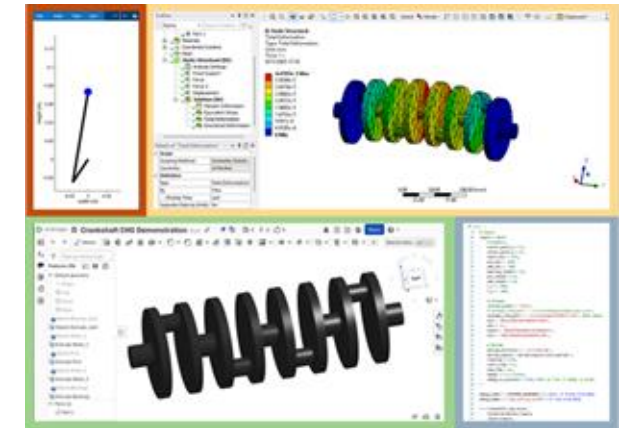
CHG Definition



Declarative Simulation



Applications



Introduction to CHGs

Universal System Simulation via Constraint Hypergraphs

Information is what we can distinguish about a system

Economy (L/km)

Color



Fuel Capacity (L)

Name

“A system is a collection of variables” —Ross Ashby

A state is data that must be temporally consistent

Economy (L/km)
10
11
12

Color
Blue
Gray
Tan



Fuel Capacity (L)
100,000
115,000
130,000

Name
C-17 Globemaster
C-5 Galaxy
C-130 Hercules

Systems are described by either observations or simulations

Economy (L/km)

10

11

12

Color

Blue

Gray

Tan

Fuel Capacity (L)

100,000

115,000

130,000

Name

C-17 Globemaster

C-5 Galaxy

C-130 Hercules



Observations are measurements we make of reality

Economy (L/km)
10
11
12

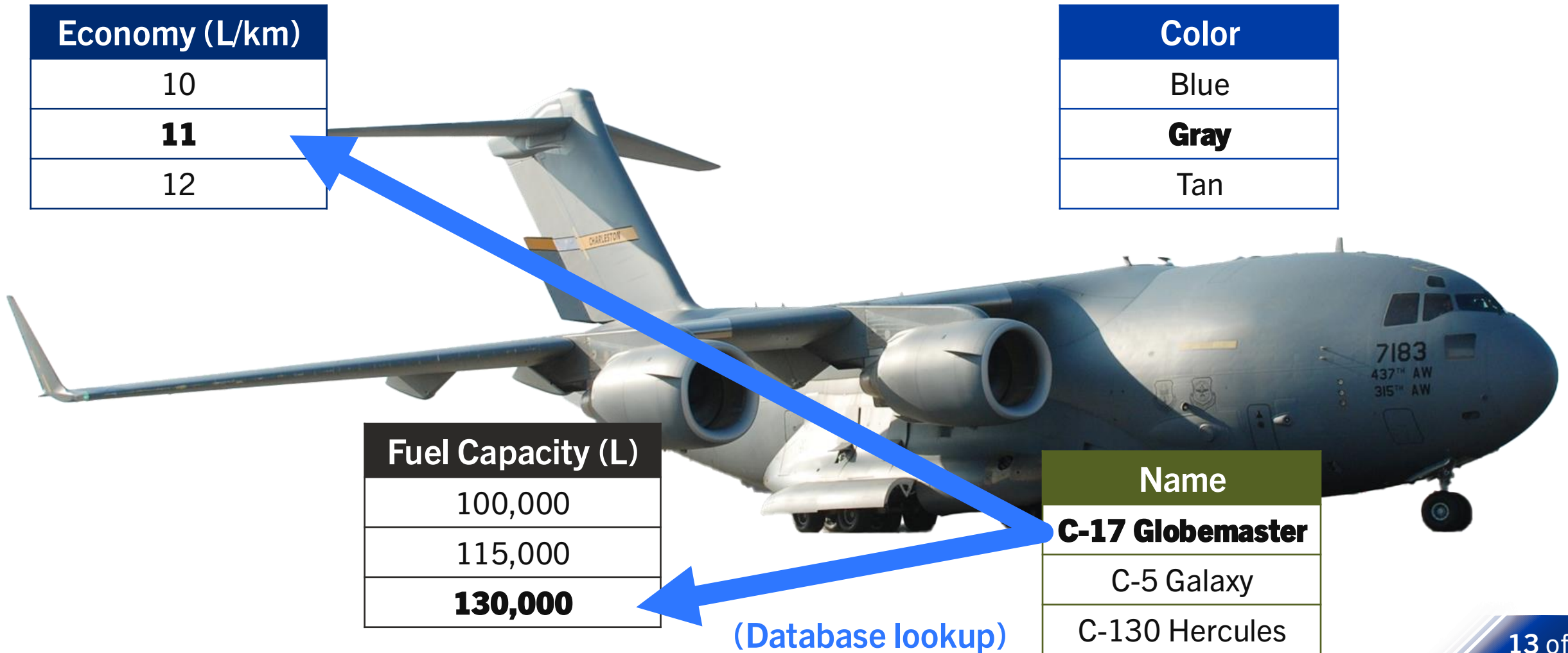
Color
Blue
Gray
Tan



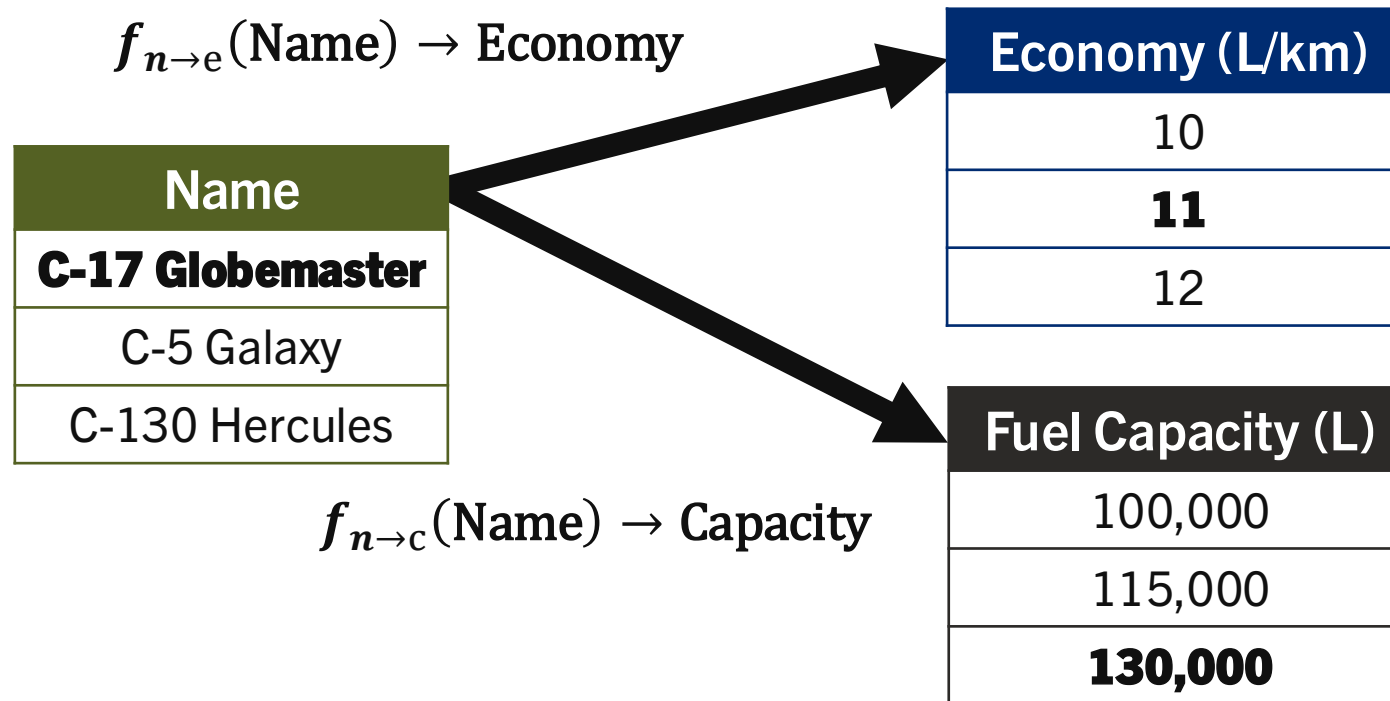
Fuel Capacity (L)
100,000
115,000
130,000

Name
C-17 Globemaster
C-5 Galaxy
C-130 Hercules

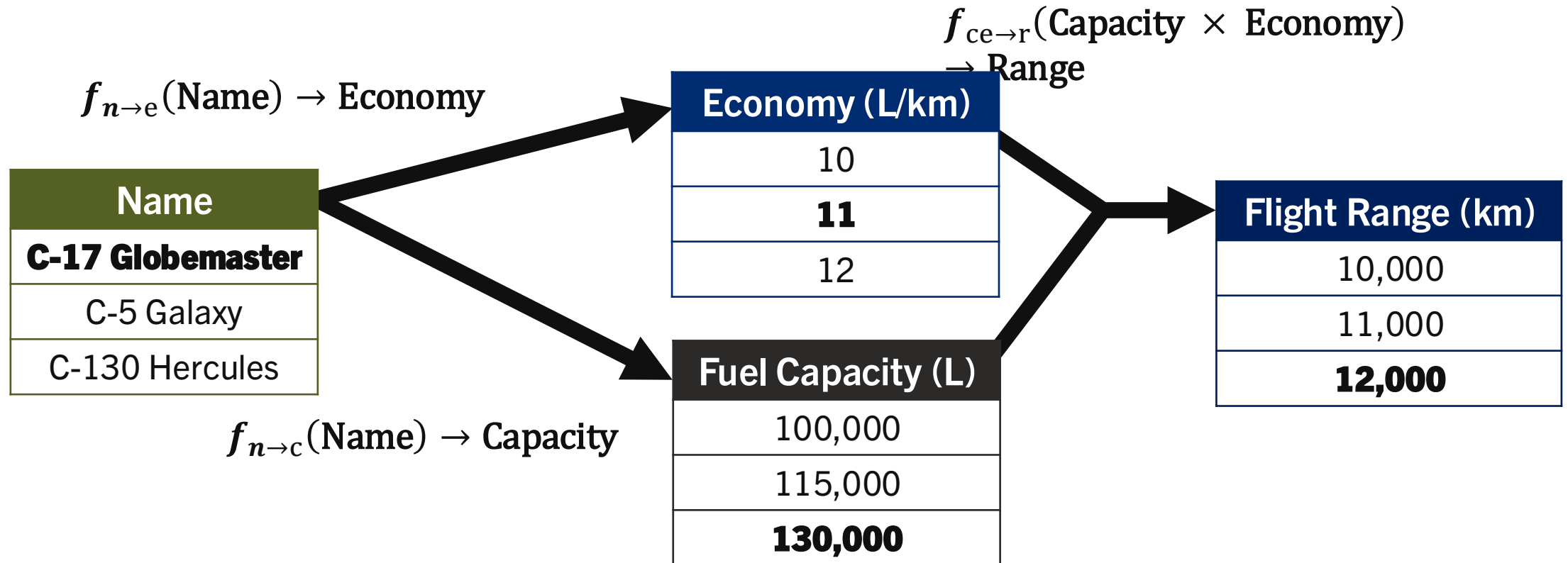
Simulations are predictions we make based on relationships



Model relations are algebraic functions that describe behavior

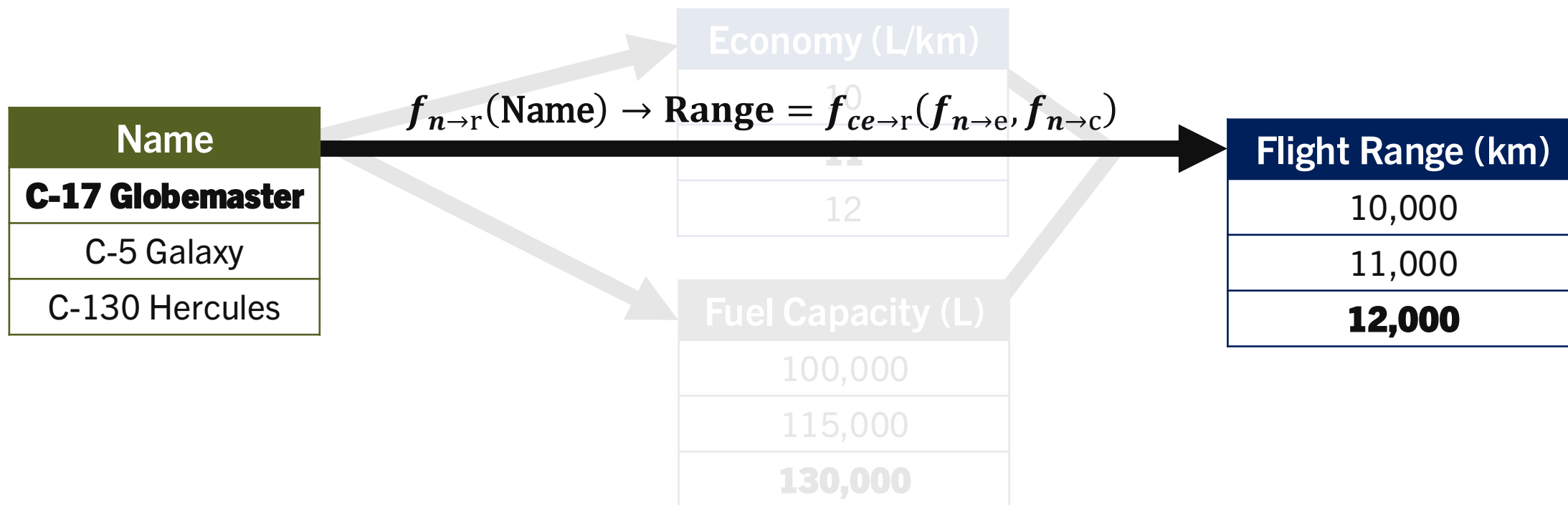


Functions can have multiple arguments

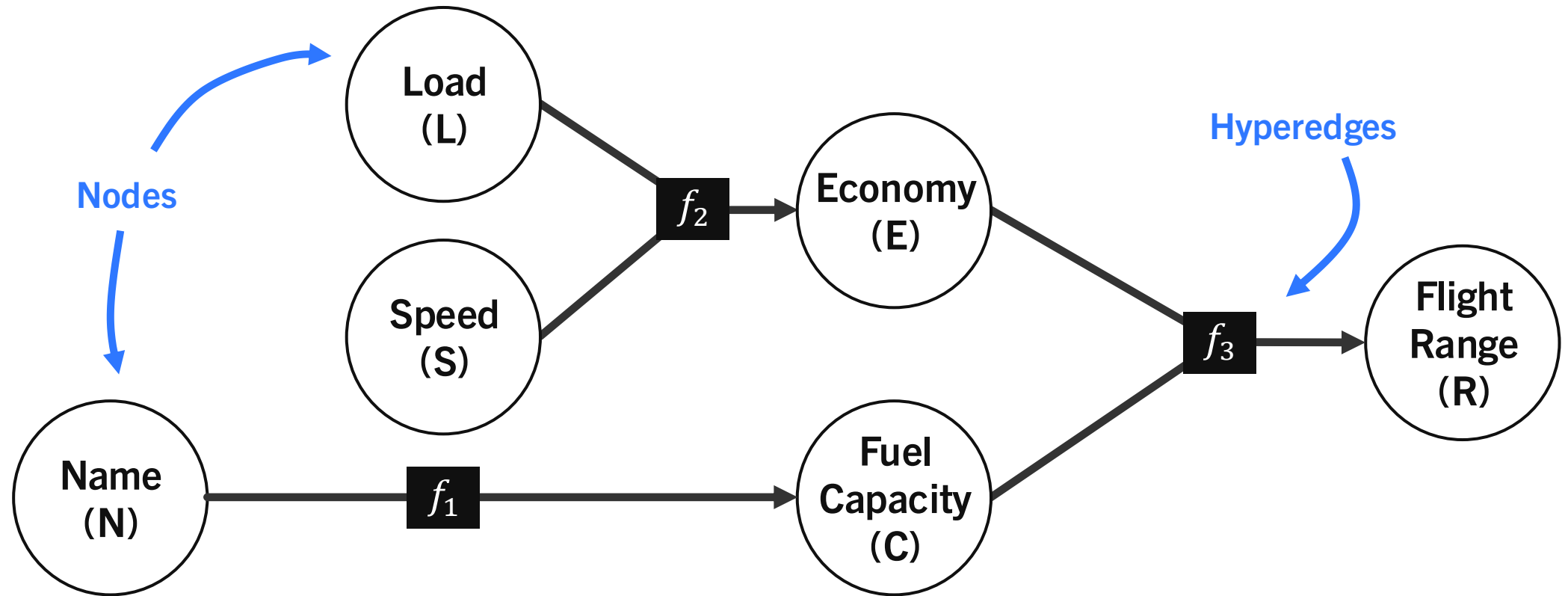


$f_{n \rightarrow e}(\text{Name}) \rightarrow \text{Economy}$

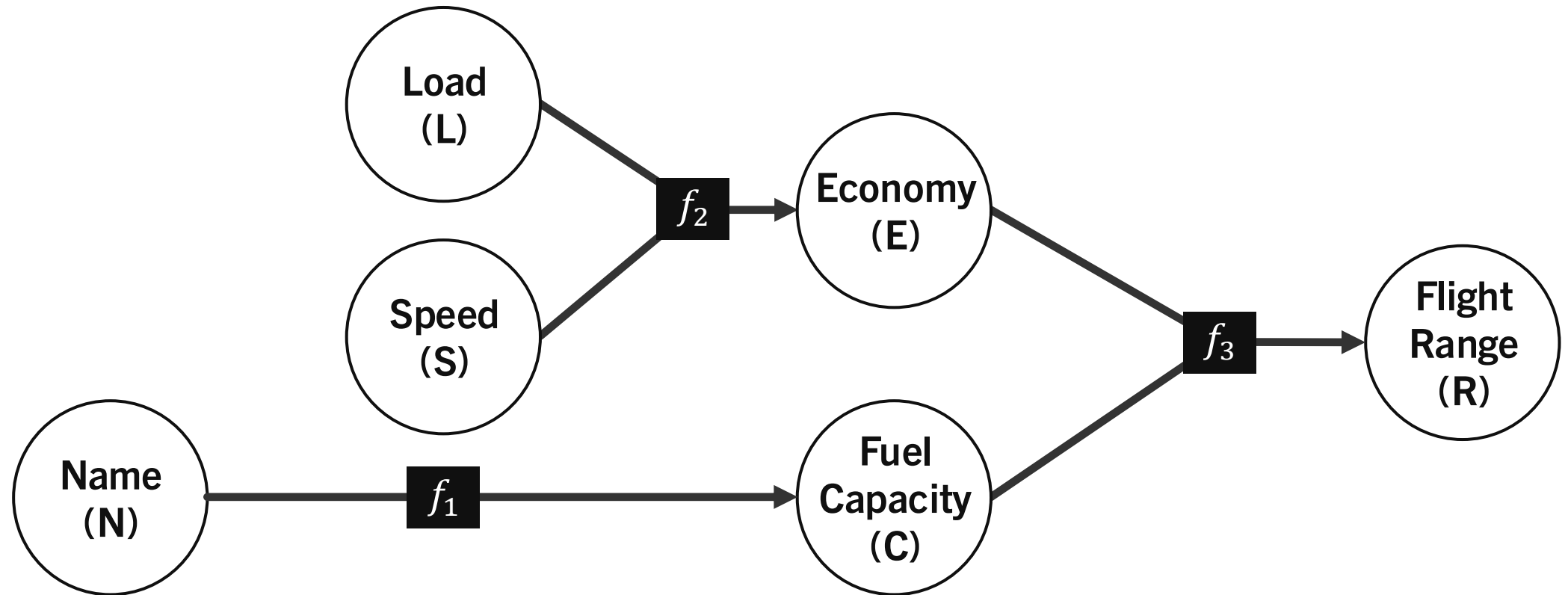
Functions compose to form greater functions



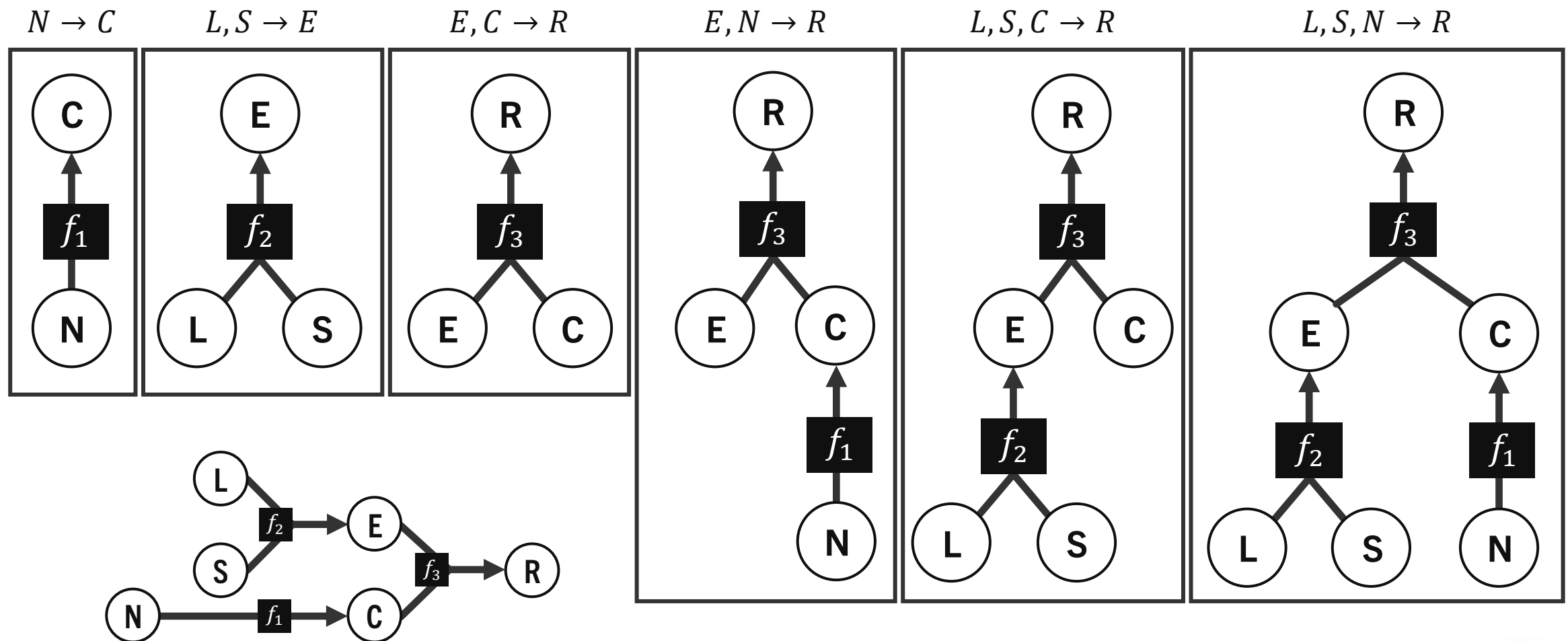
We can represent all models and variables as a constraint hypergraph (CHG)



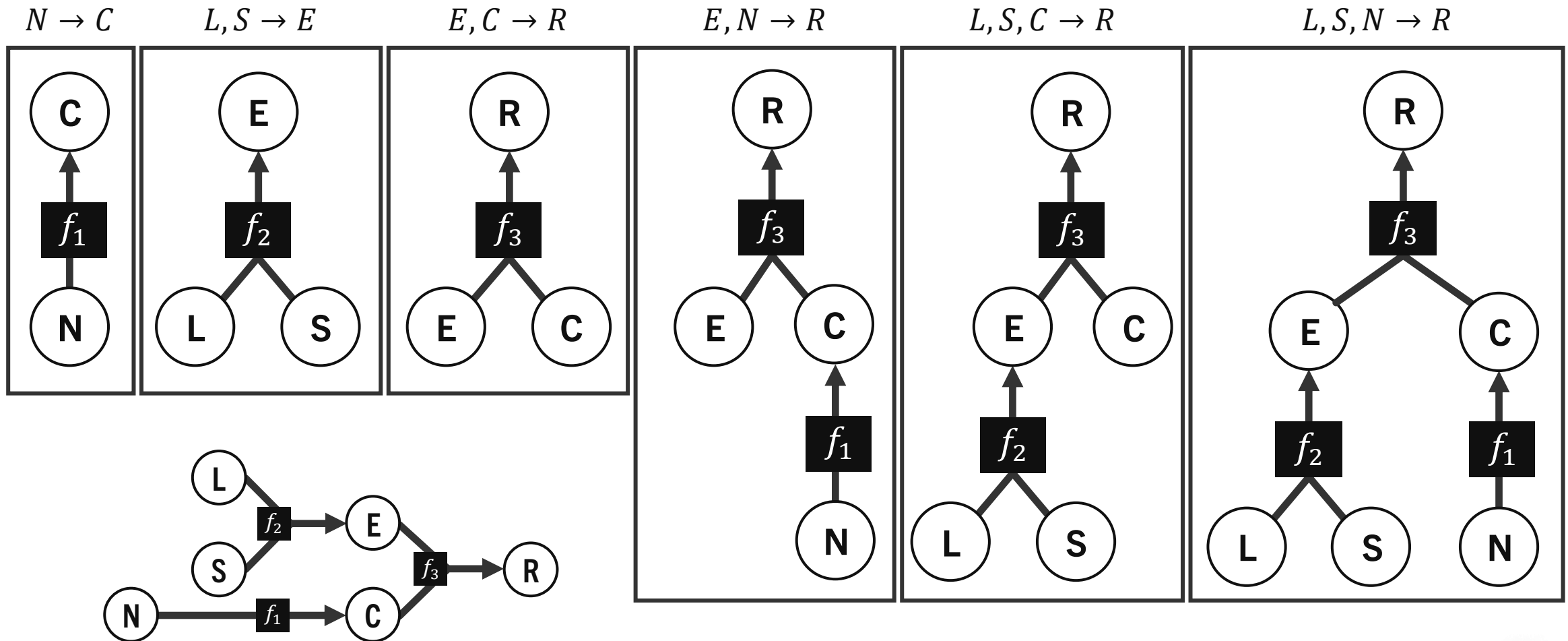
Every path in a CHG is a simulation



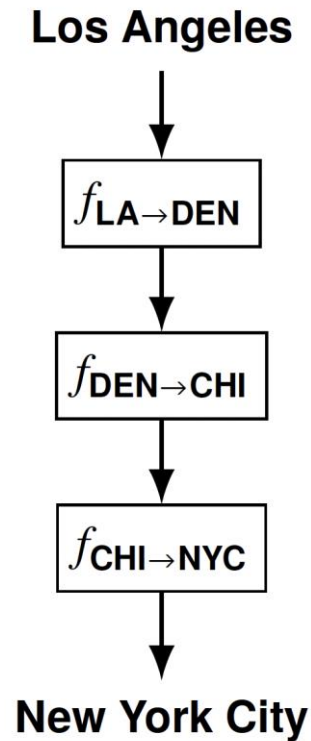
Every path in a CHG is a simulation



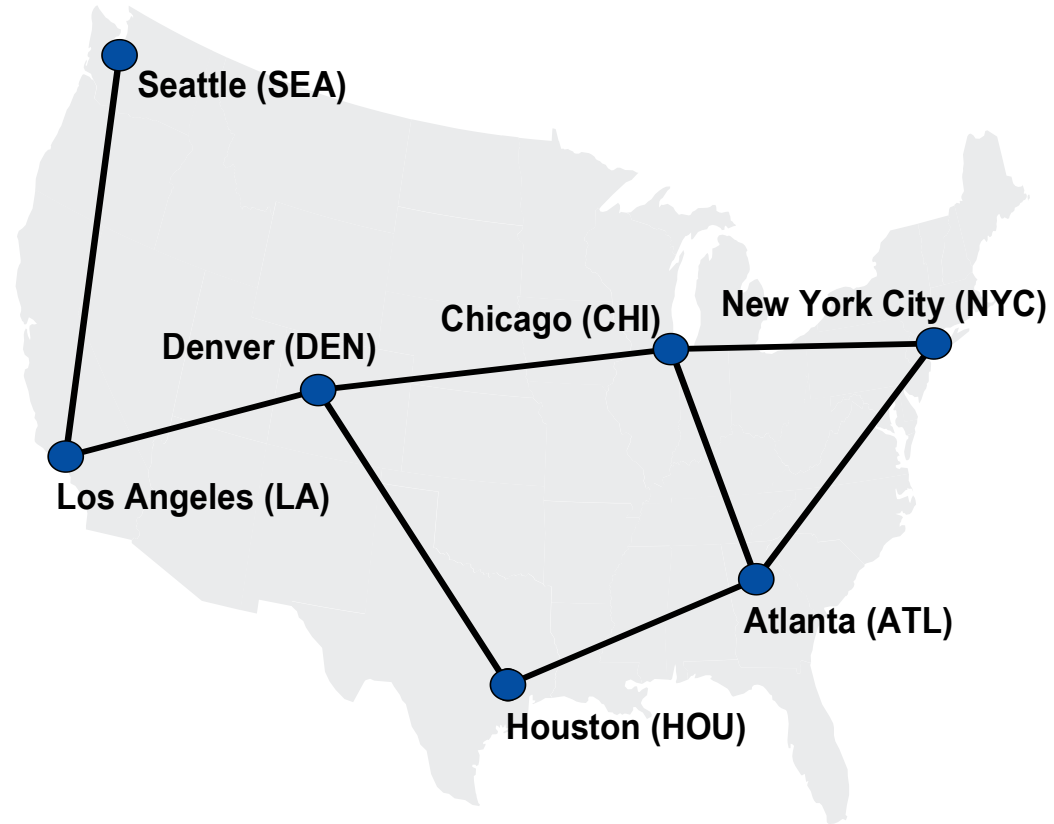
Embedded behavior (paths) enables declarative simulation



Declarative models capture the semantics of the system

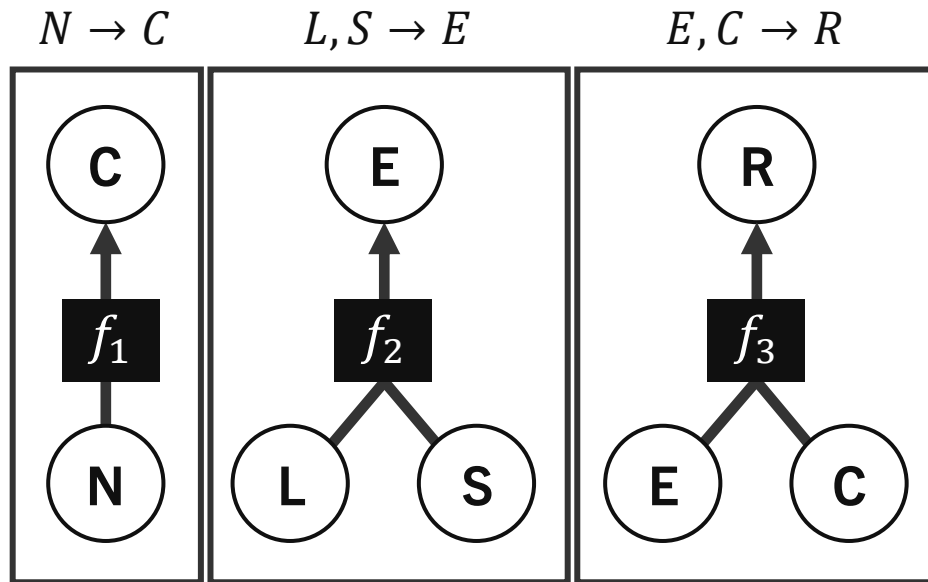


Imperative

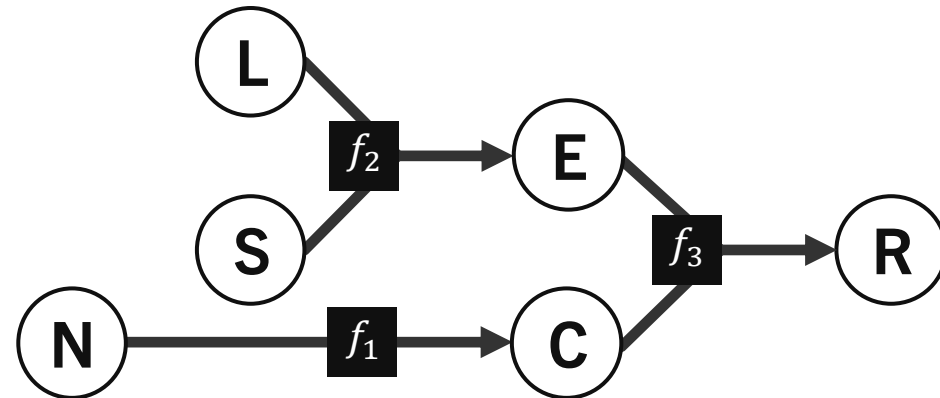


Declarative

Declarative simulation in a CHGs is pathfinding



Imperative

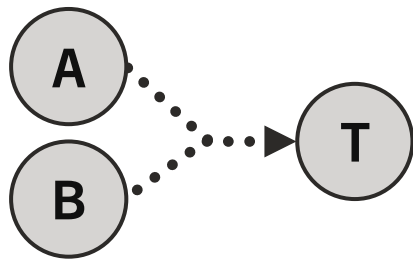


Declarative

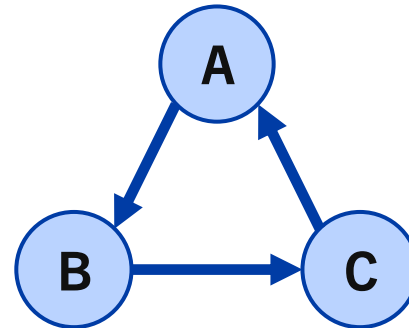
Structure of a CHG

Universal System Simulation via Constraint Hypergraphs

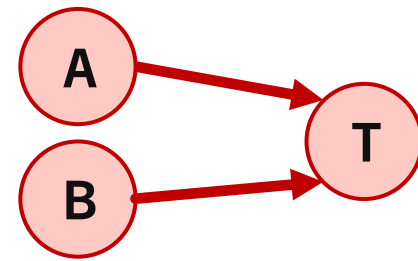
There are three structures in a CHG that a declarative agent needs to process:



Partial Edges

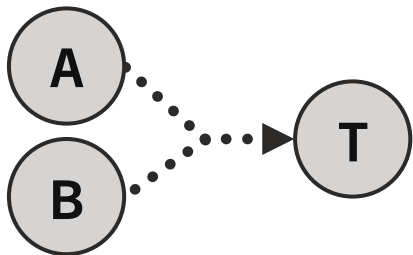
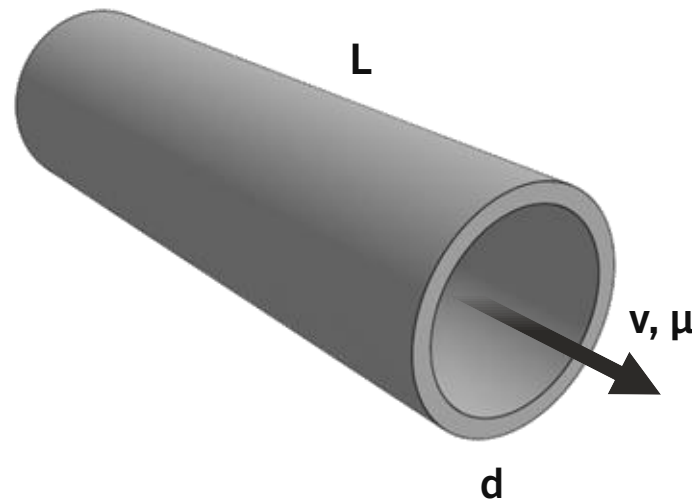


Cycles



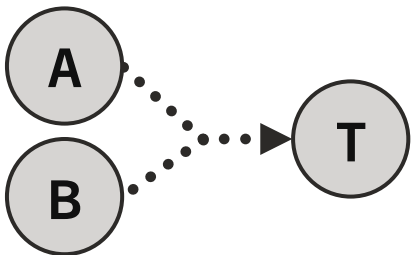
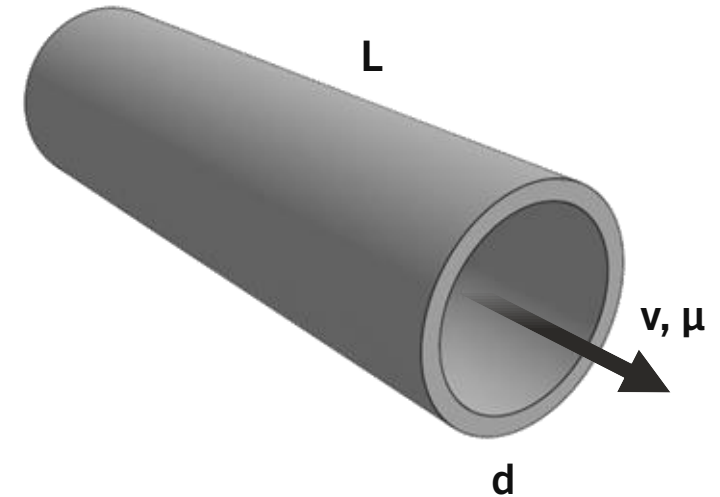
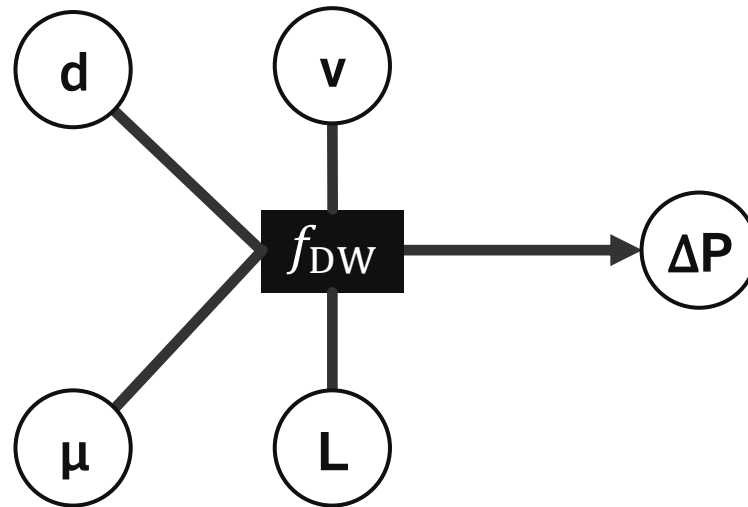
Multi-Edges

Partial edges occur when we don't have an explicit mapping for every variable



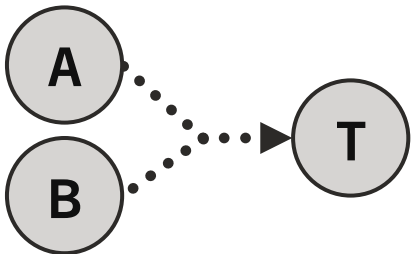
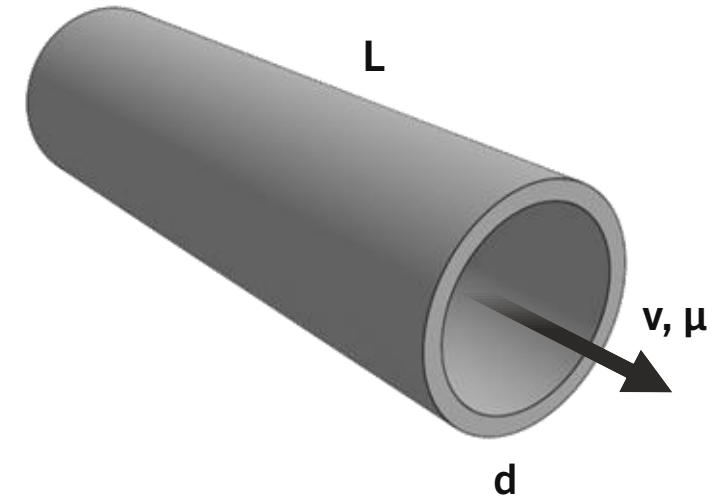
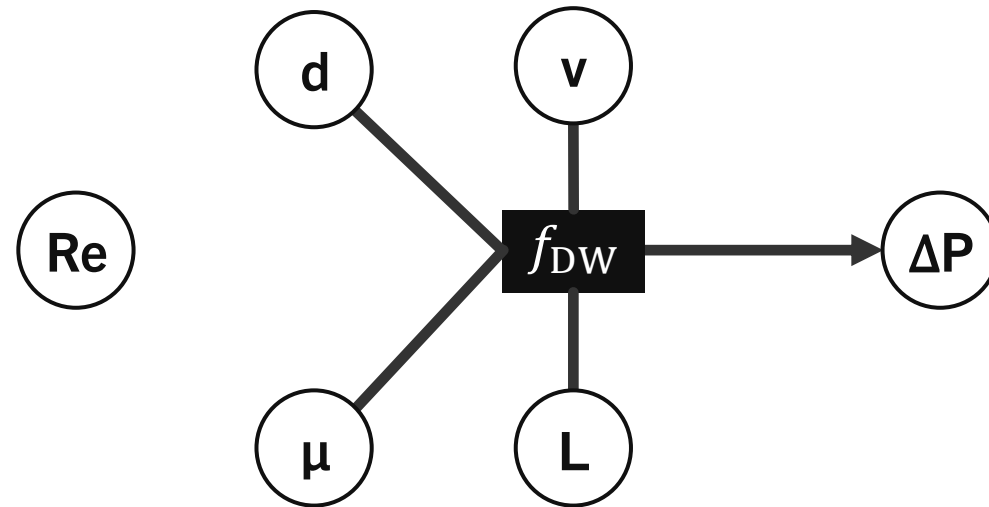
Goal is to calculate pressure loss from fluid flow in a pipe

Partial edges occur when we don't have an explicit mapping for every variable



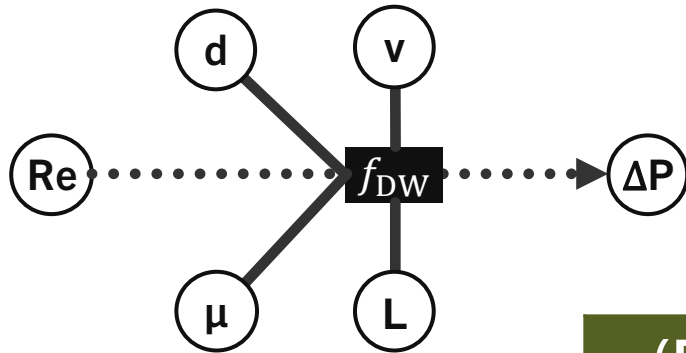
The hypergraph captures the Darcy-Weisbach relationship $\Delta P = \frac{32\mu v L}{d^2}$

Partial edges occur when we don't have an explicit mapping for every variable

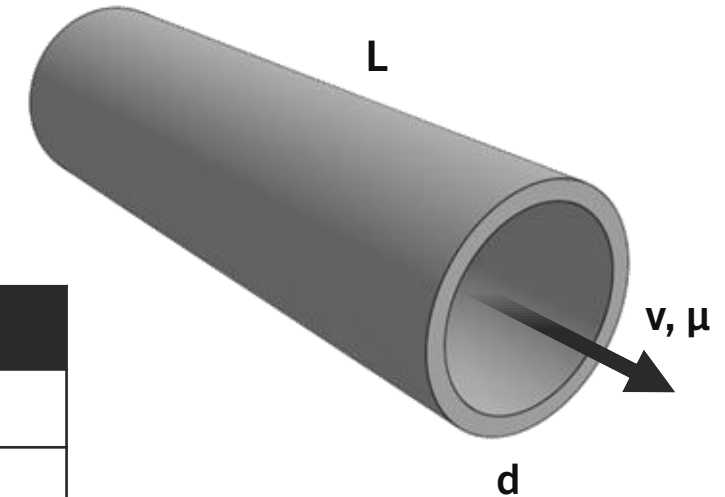
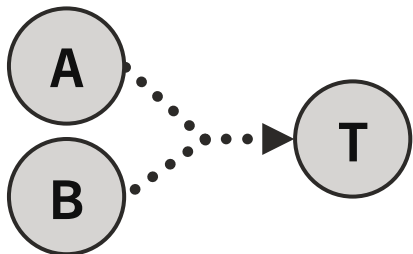


This relationship is only valid if the flow is laminar ($Re < 2300$)

Partial edges occur when we don't have an explicit mapping for every variable

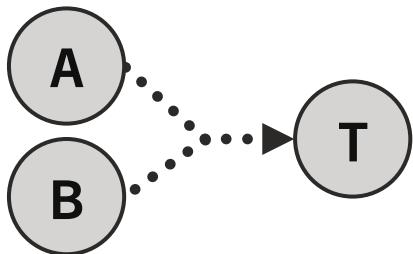
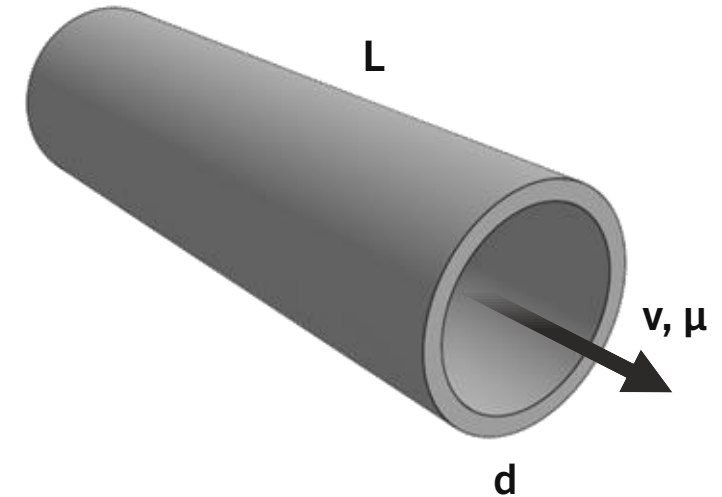
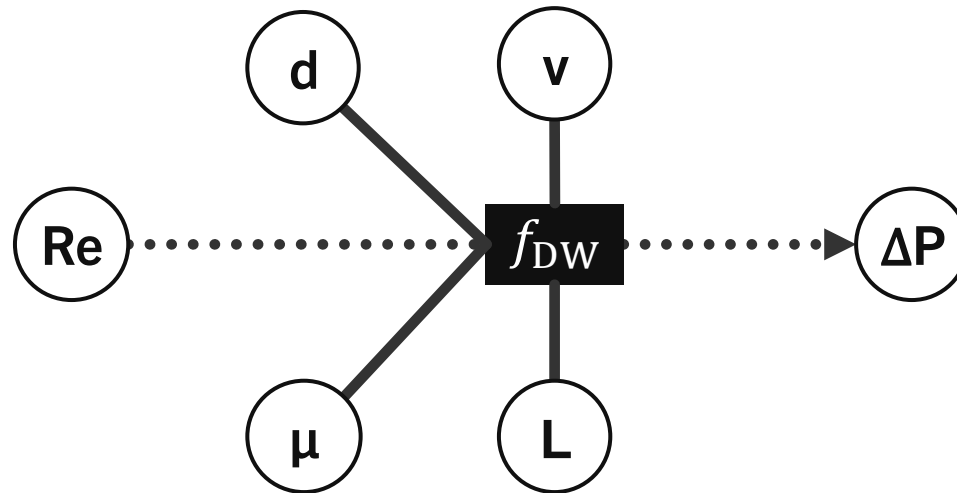


(Re, L, μ, d, v)	f_{DW}	ΔP
...		...
$(1000, L, \mu, d, v)$	→	0.5
$(2000, L, \mu, d, v)$	→	1
$(3000, L, \mu, d, v)$		1.5
...		...



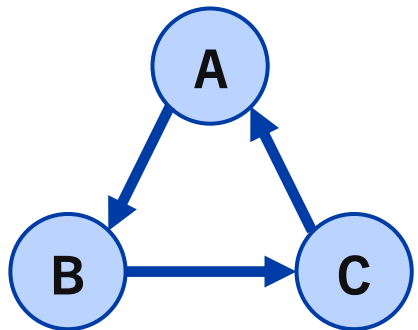
We show this by mapping a subset of the domain

Partiality means that functions don't automatically compose



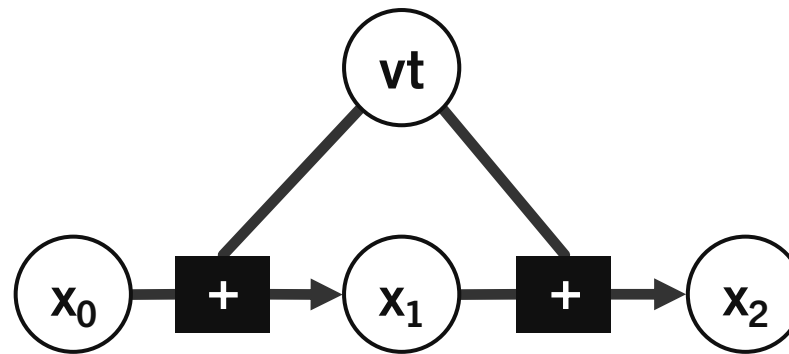
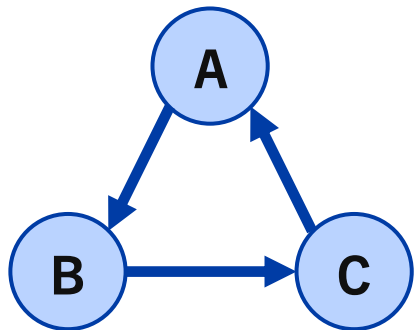
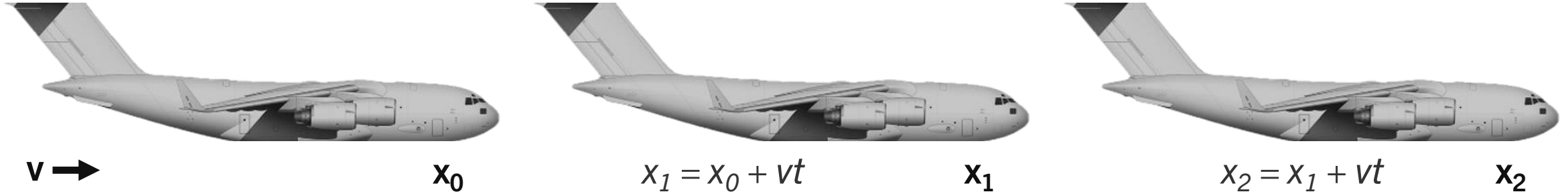
The dotted line shows that our function doesn't handle every value of Re

Cycles indicate behavioral patterns



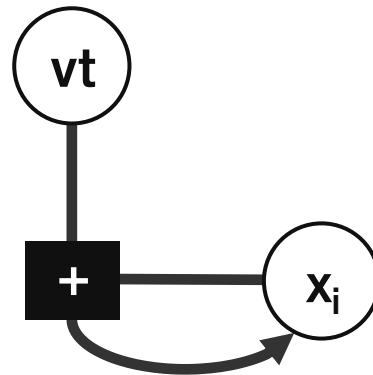
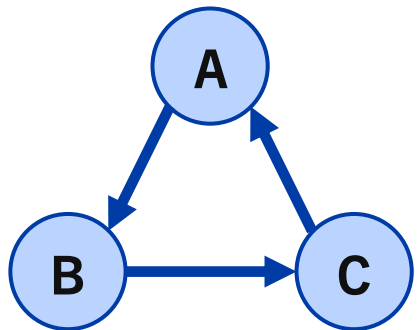
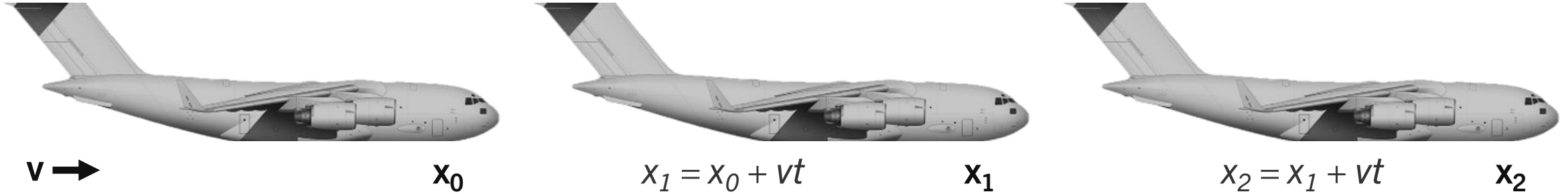
Plane moving at constant velocity

Cycles indicate behavioral patterns



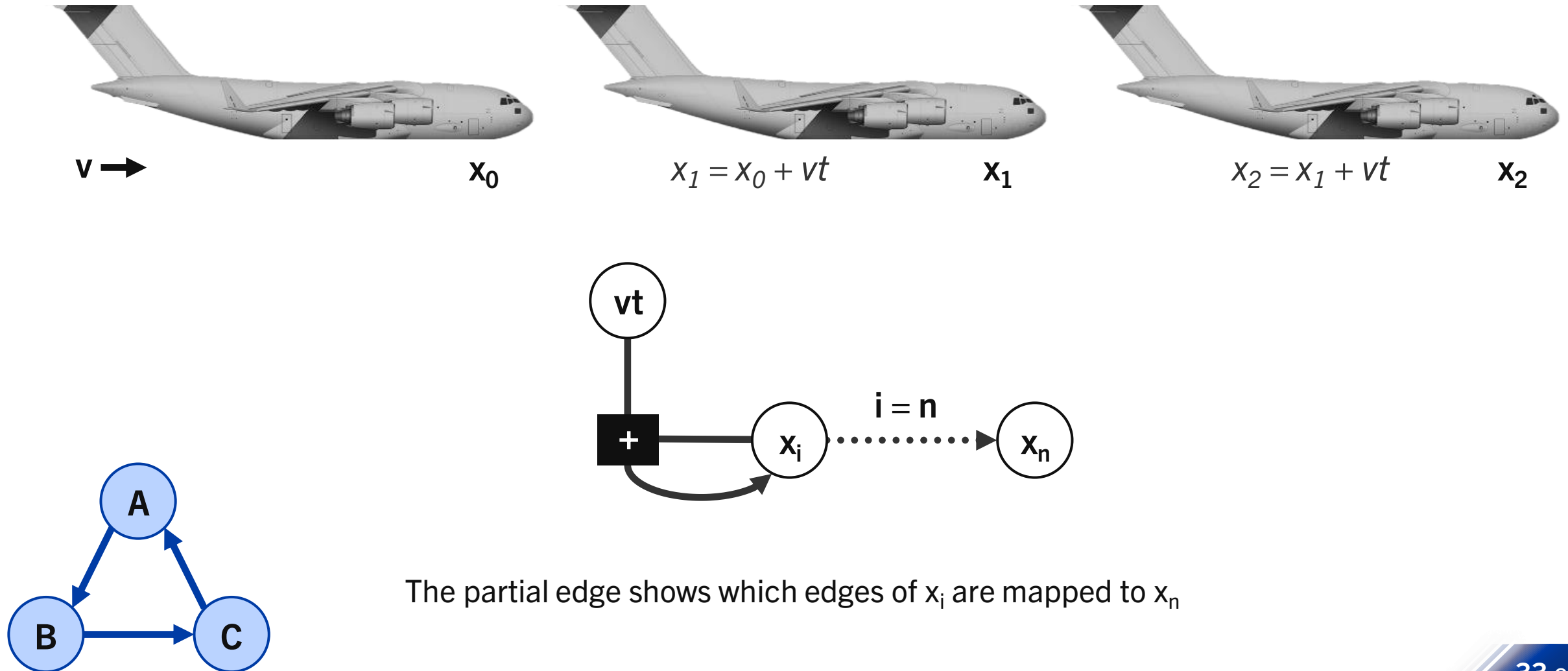
x_0 , x_1 , and x_2 are all different nodes

Cycles indicate behavioral patterns



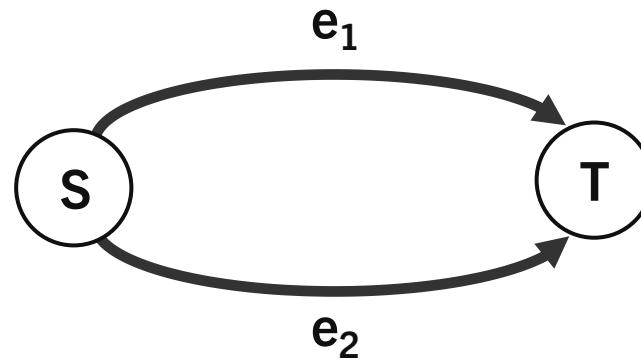
A more expressive way to show this is $x_{i+1} = x_i + vt$ (unraveling)

Unraveling requires an exit condition to be solvable

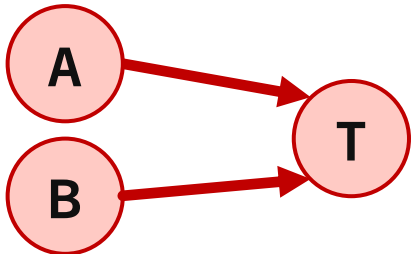


Multi-edges indicate competing models

 **Ansys**

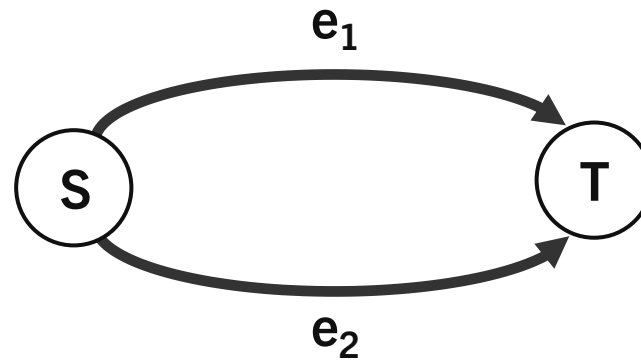


 ChatGPT

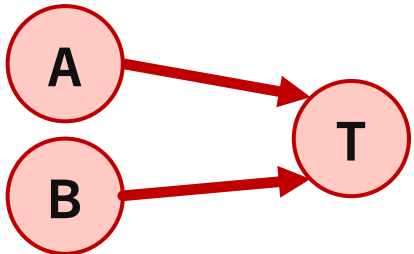


Both edges are viable (“correct”), but only one can be selected for simulation

Ansys



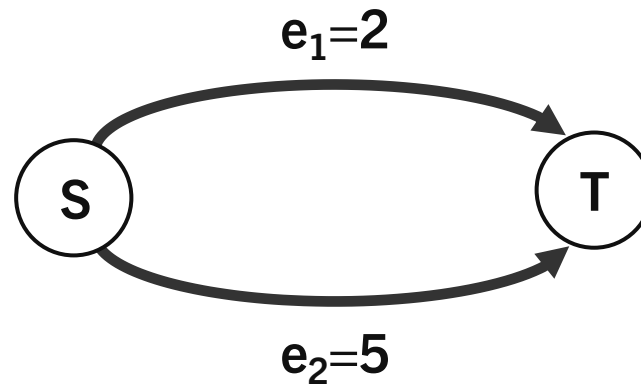
 ChatGPT



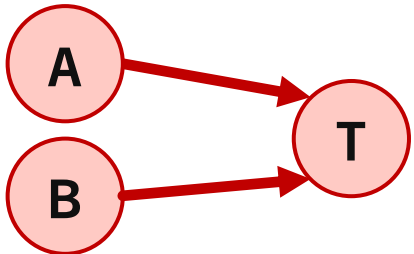
Differences might be on accuracy, computation time, software availability, etc.

Weights allow the agent to *prefer* one edge to another

Ansys



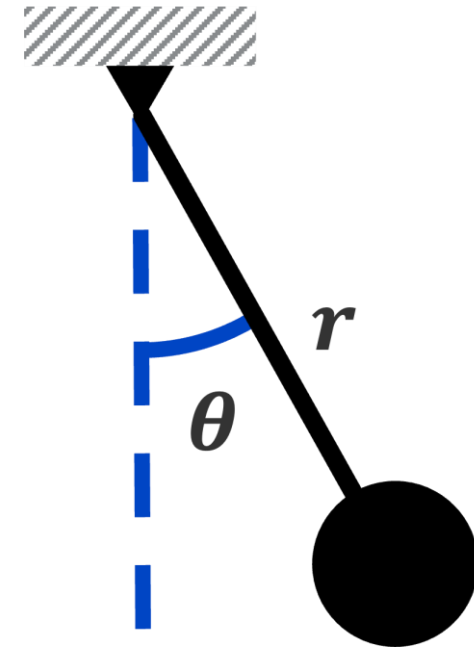
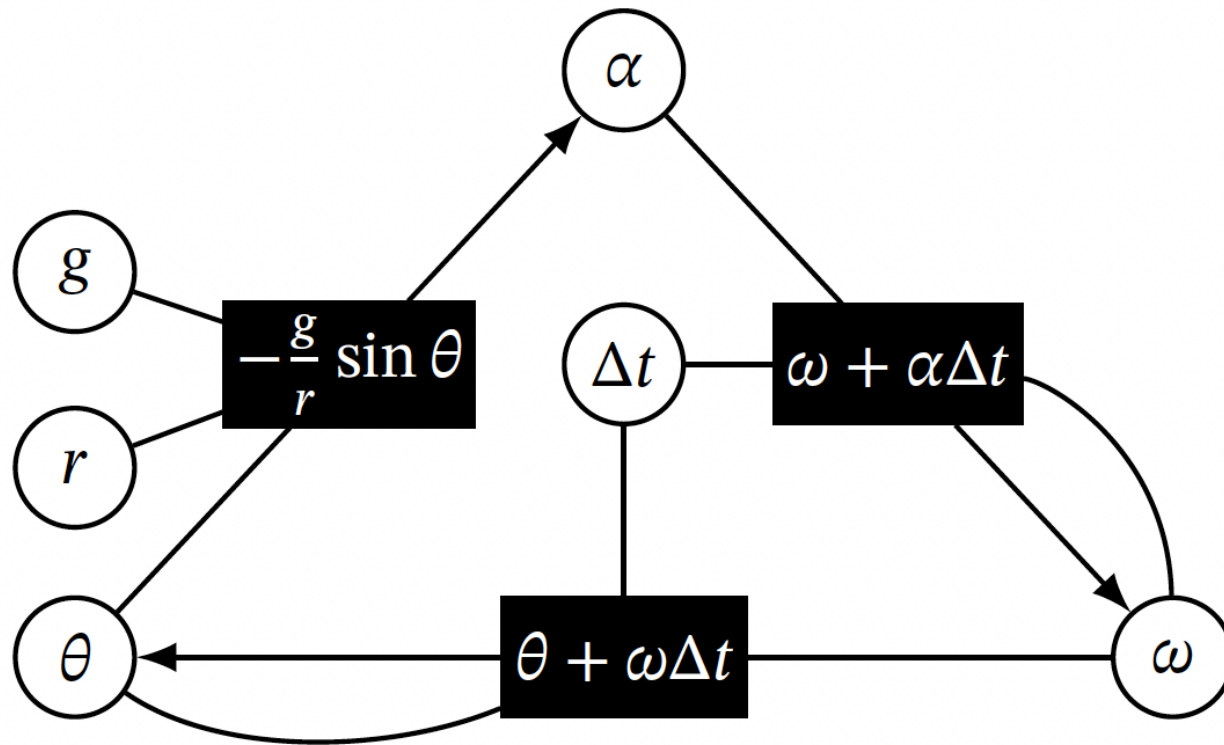
ChatGPT



Demonstration

Universal System Simulation via Constraint Hypergraphs

Pendulum System



Declarative simulation provided by ConstraintHg





ConstraintHg

constrainthg 0.3.2

```
pip install constrainthg
```

Kernel for building and simulating constraint hypergraphs.

Navigation

[Project description](#)
[Release history](#)
[Download files](#)

Project description



JOSS 10.21105/joss.09131

docs passing

test/linter passing

release v0.3.2

last commit february



SOFTWARE

[Quickstart](#)
[Tutorial](#)
[API](#)
Node
Edge
Hypergraph
Relations Module
[About](#)
[Demos](#)
[Repository](#)

CONSTRAINT HYPERGRAPHS

[CHG Overview](#)
[Learn About CHGs](#)

Quickstart

Use [PIP](#) to install ConstraintHg into your Python environment:

```
pip install constrainthg
```

From there you'll want to import the library into your Python script. This is a pretty typical method to use:

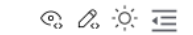
```
from constrainthg.hypergraph import Node, Hypergraph
import constrainthg.relations as R
```

Simple Demo

Note that this demo is found in [demos/demo_basic.py](#)

Let's build a basic constraint hypergraph of the following equations:

- $A + B = C$
- $A = -D$
- $B = -E$
- $D + E = F$
- $F = -C$

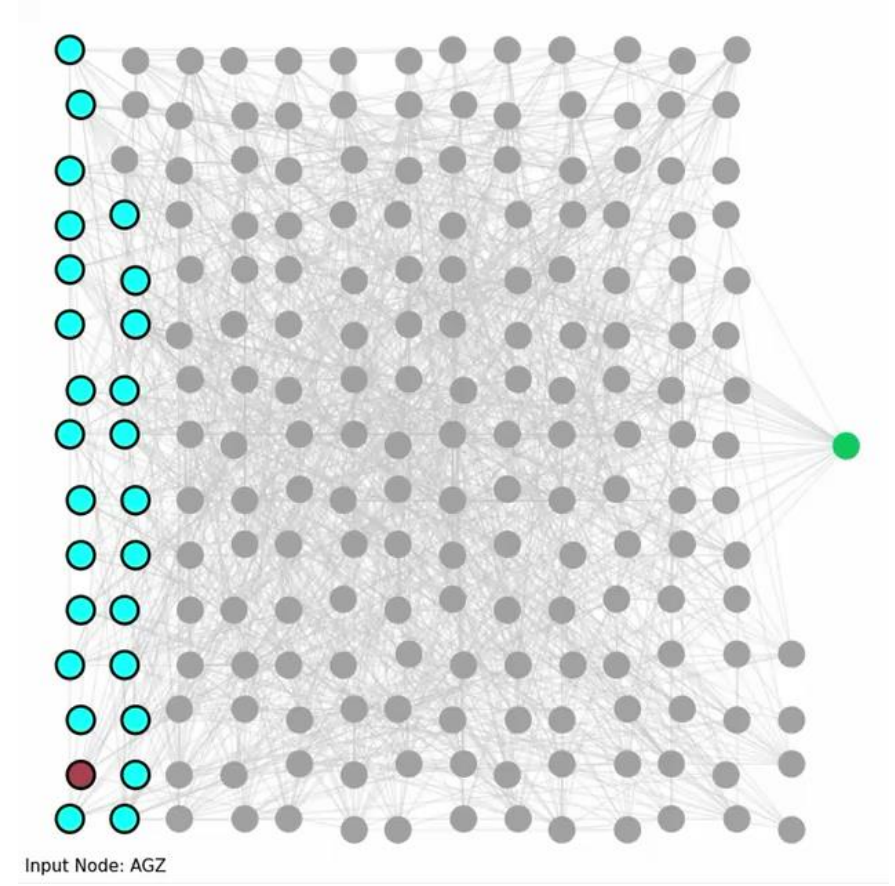


ConstraintHg Functionality



Provides methods for:

- Representing and visualizing CHGs
- Forming cycles, partial edges, and edge weights
- Pathfinding for declarative simulation
 - Extensive logging tools
- Merging CHGs



Process of making a CHG

1. Identify system facts (nodes)

Parameters, application variables, API tokens, etc.



2. Form relations (edges) between nodes

Relations show how one node is determined by a set of other nodes



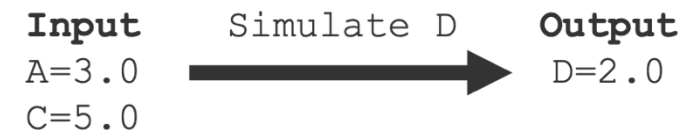
3. Pass to CHG solver

Solver parses the CHG



4. Request simulation

Solver simulates requested output by finding the shortest path mapping it to a set a known inputs



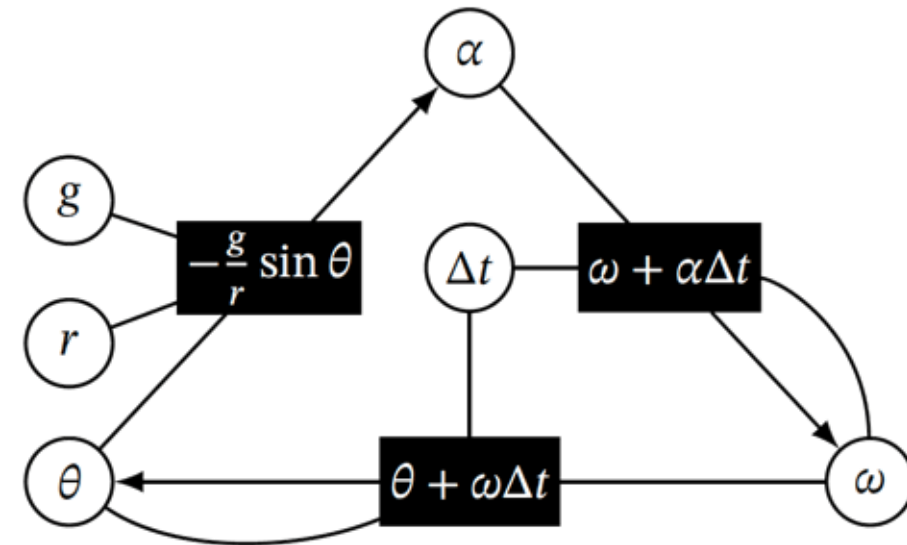
Declarative simulation requires only start & end points

```
hg.solve(  
    target='theta',  
    inputs=(  
        theta=0.785,  
        omega=0.0,  
        g=9.81,  
        r=0.25,  
        delta_t=0.02  
    ),  
    min_index=2,  
)
```

Simulation call

```
└─theta(2)=0.7739  
   └─omega(2)=-0.5547  
      └─alpha(2)=-27.74  
         └─g=9.81  
            └─theta=0.785  
               └─r=0.25  
                  └─delta_t=0.02  
                     └─omega=0  
                        └─delta_t=0.02  
                           └─theta=0.785
```

Discovered, unraveled simulation



```

1 """
2 Demonstration of ConstraintHg for a simple pendulum.
3 More examples and information available online at constrainthg.readthedocs.io
4 """
5

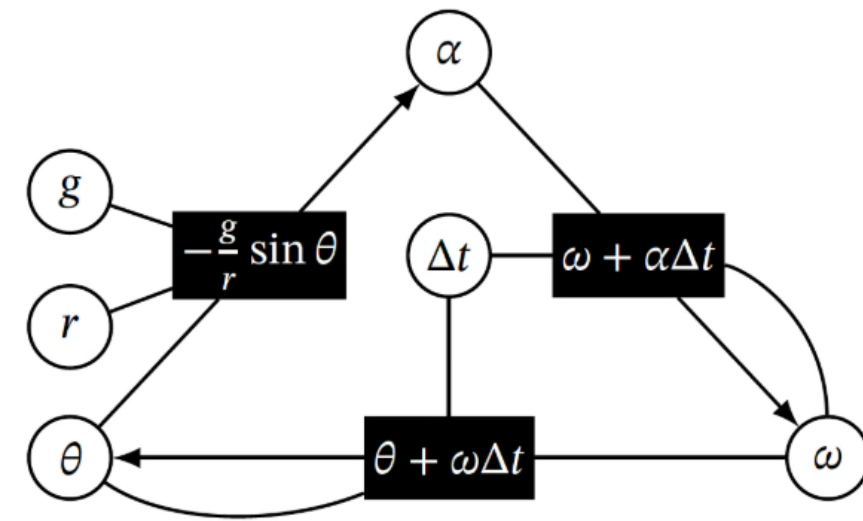
```

Run Cell | Run Below | Debug Cell

```

6 # %%
7 # Setup constraint hypergraph
8 from constrainthg import Hypergraph
9 from math import sin
10
11 # Define relational rules as methods in Python
12 def Rtheta_to_alpha(theta, g, r):
13     return -g / r * sin(theta)
14
15 def Rintegrate(initial, slope, step):
16     return initial + slope * step
17
18 def Rcheck_increment(initial, slope):
19     return slope == initial + 1
20
21 # Initialize constraint hypergraph and add edges
22 hg = Hypergraph()
23
24 hg.add_edge(
25     sources=dict(theta='theta', g='g', r='r'), target='alpha',
26     rel=Rtheta_to_alpha, index_offset=1,
27     label='-g/r*sin(theta)'
28 )
29
30 hg.add_edge(
31     sources=dict(initial='omega', slope='alpha', step='del_t'), target='omega',
32     rel=Rintegrate, index_via=Rcheck_increment, disposable=['initial', 'slope'],
33     label='Integrate alpha'
34 )
35

```



Interactive-1

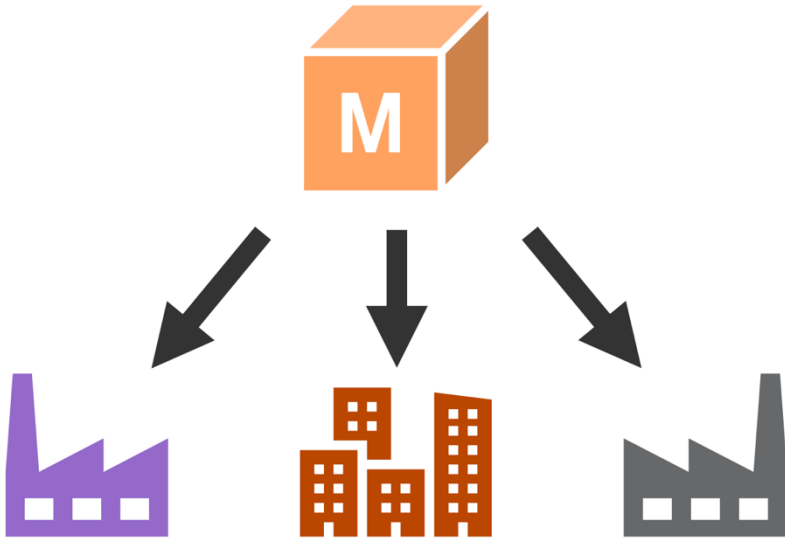
☐ Interrupt | ☒ Clear All | ☒ Restart | ☒ Jupyter Variables | ☒ Save | ... | ☒ .venv (1)

Applications of CHGs

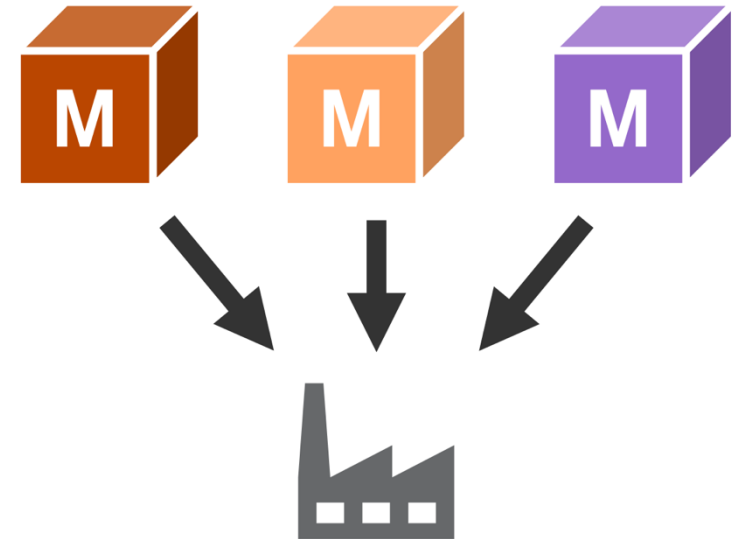
Universal System Simulation via Constraint Hypergraphs

CHGs provide platforms for model-based engineering and digital twins

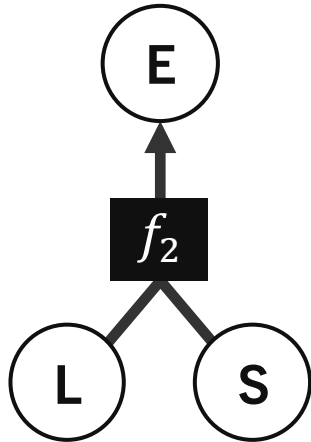
Maintains information in different contexts



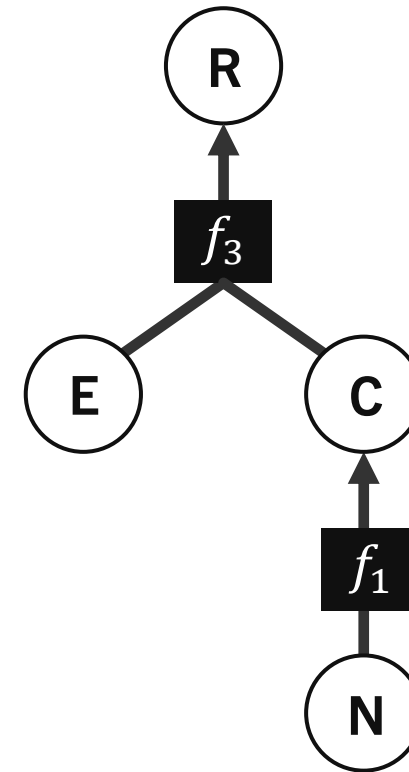
Maintains information with different models



New behaviors can be expressed as new paths in the union of two CHGs

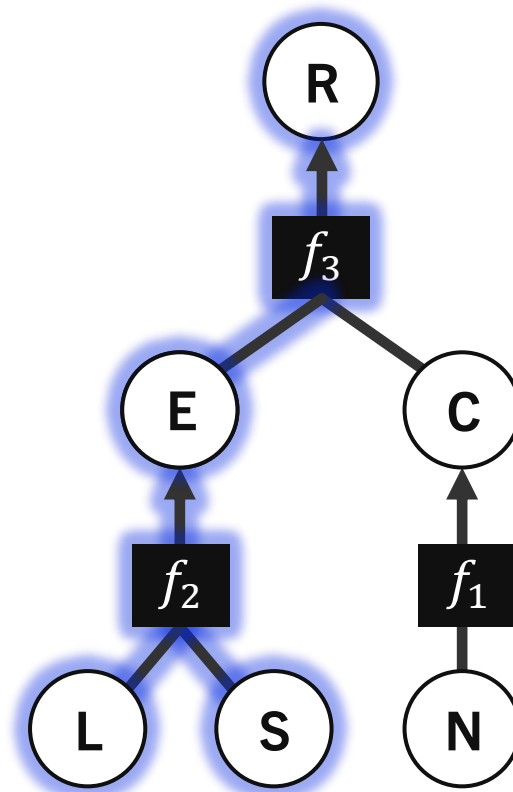


Graph 1



Graph 2

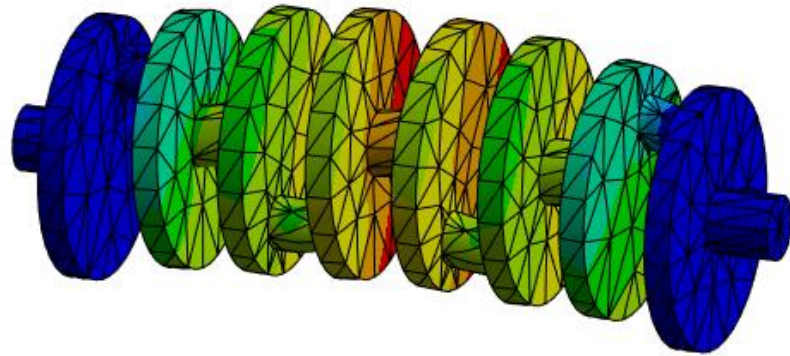
New behaviors can be expressed as new paths in the union of two CHGs



Graph 1 $\cup$ 2

New behaviors include simulation paths that go across calculating software

Material Mechanics



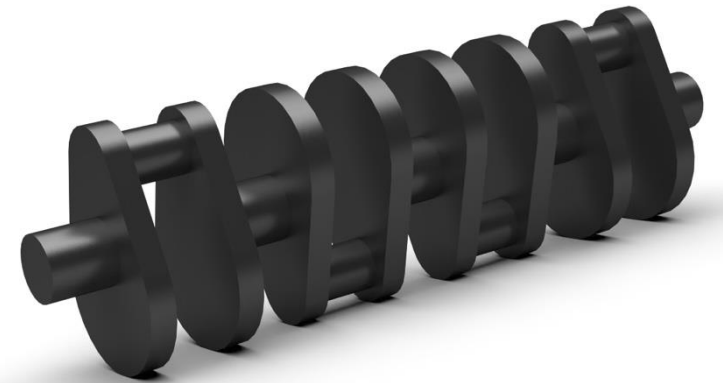
Ansys

Kinematic Analysis



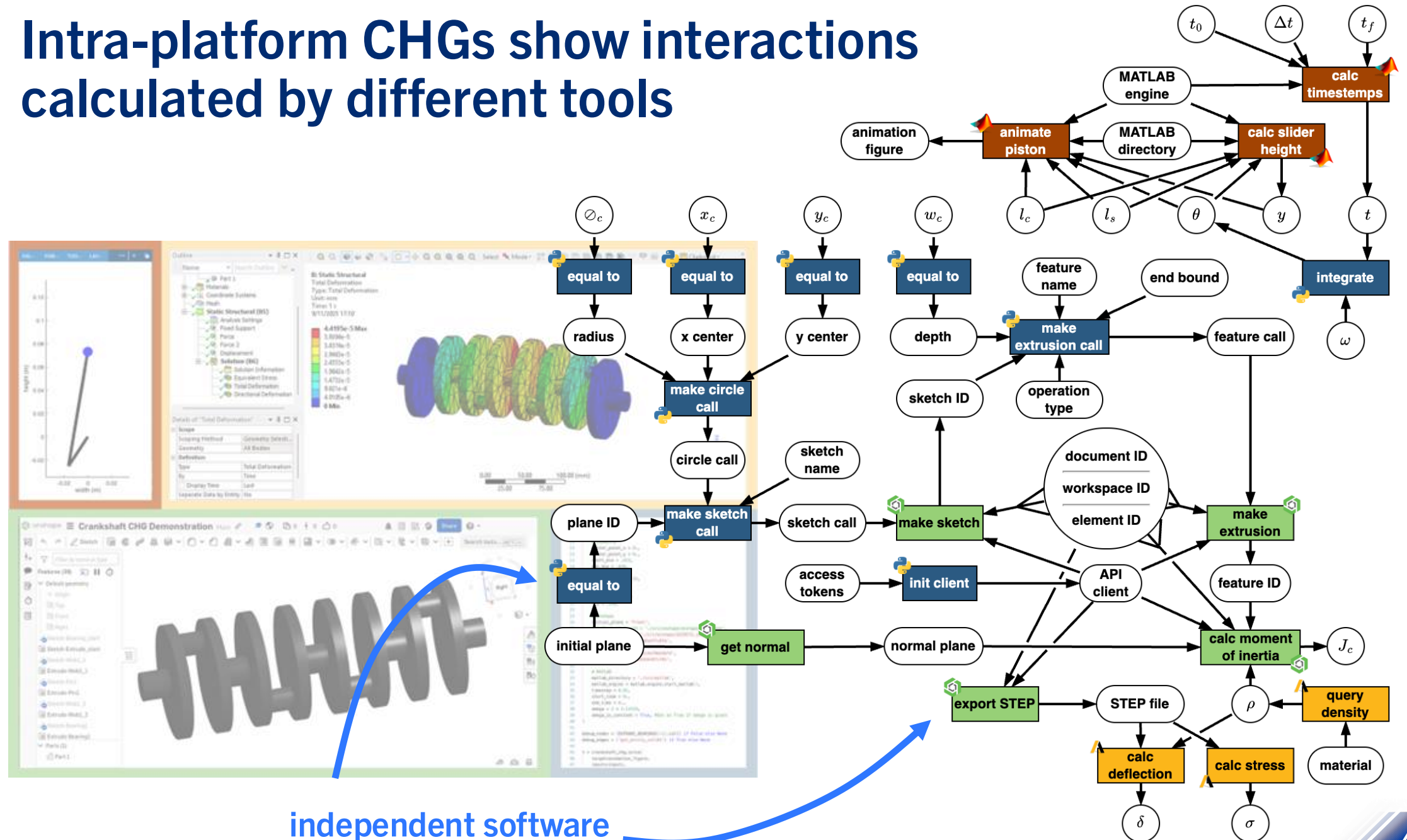
MATLAB[®]
& SIMULINK[®]

Mass Properties

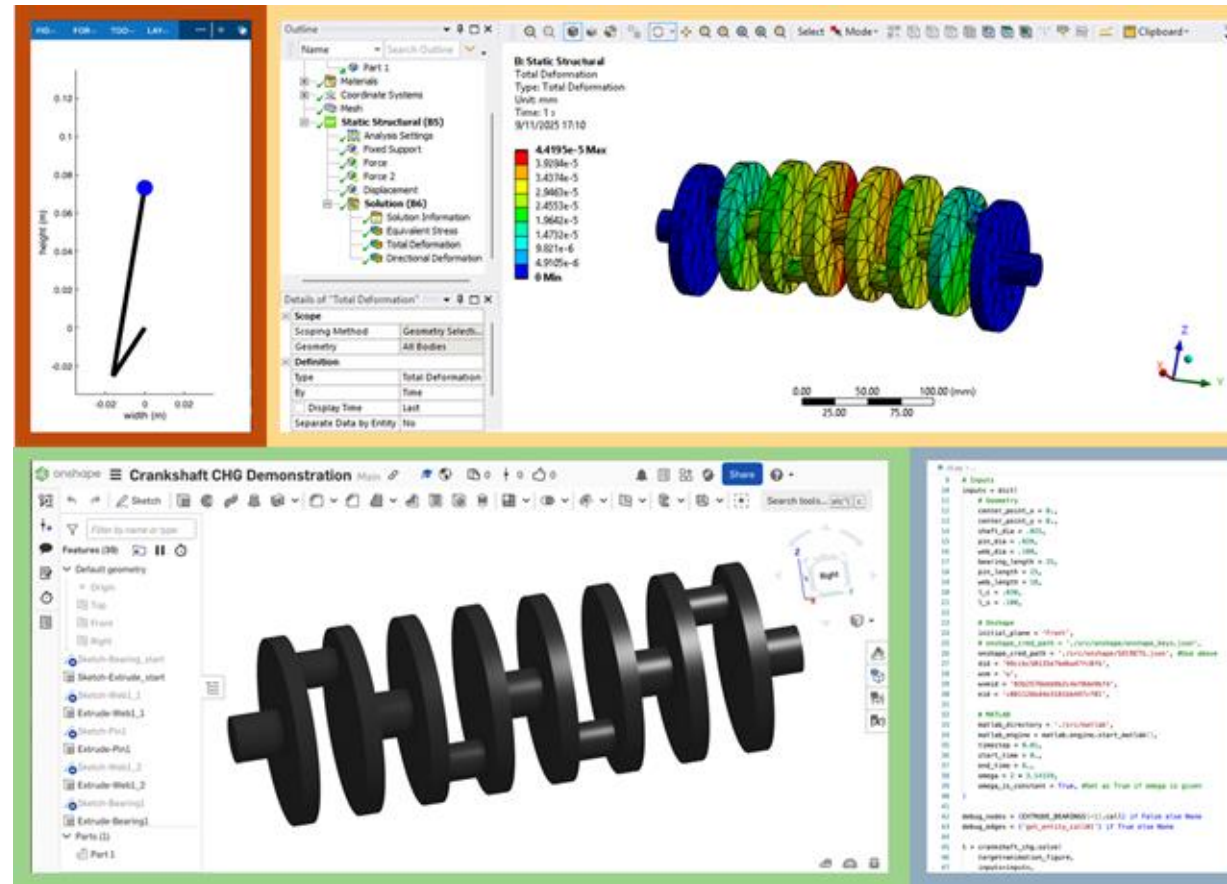


 **onshape[®]**

Intra-platform CHGs show interactions calculated by different tools



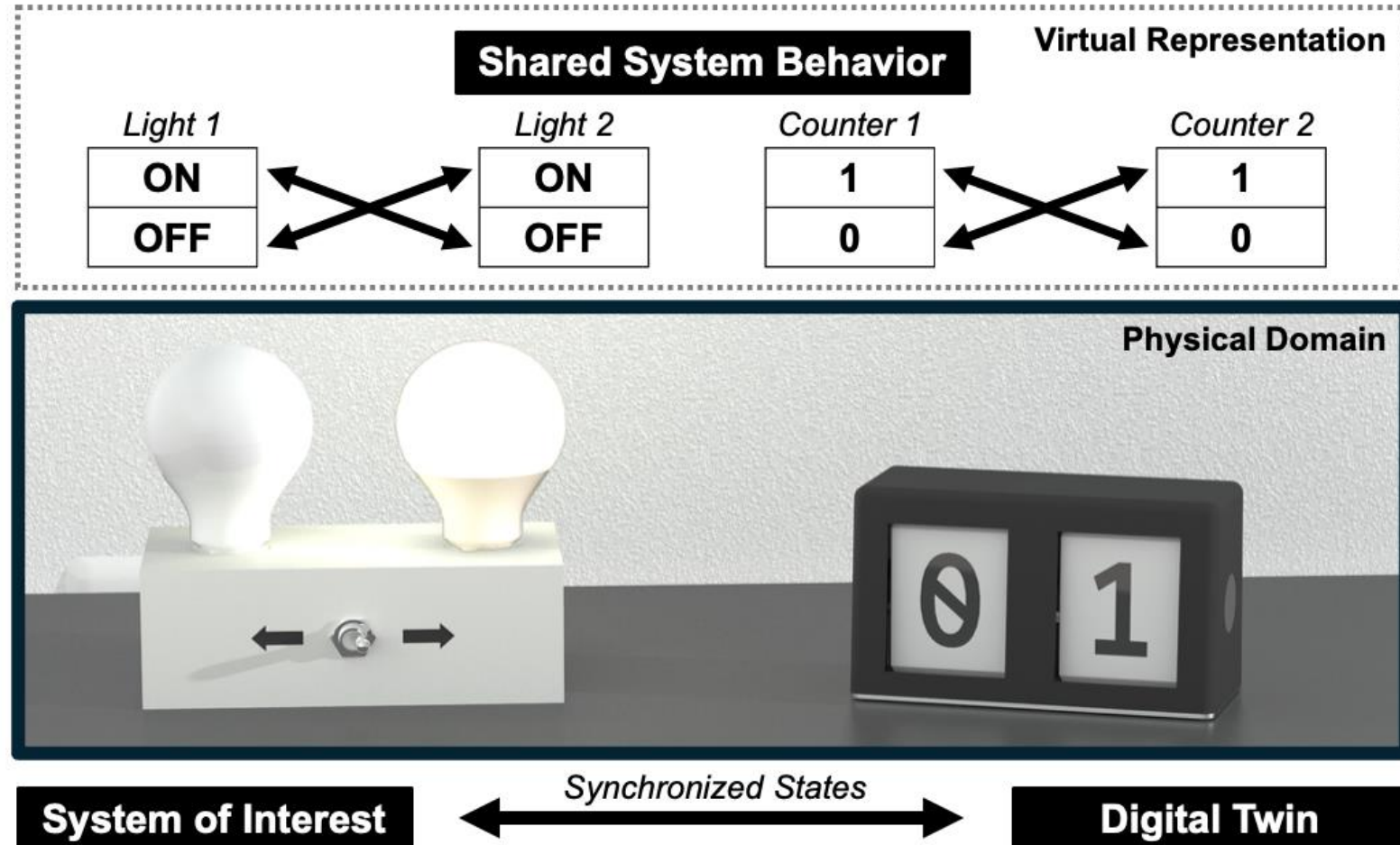
CHGs resolve inter-application APIs into a cohesive ecosystem



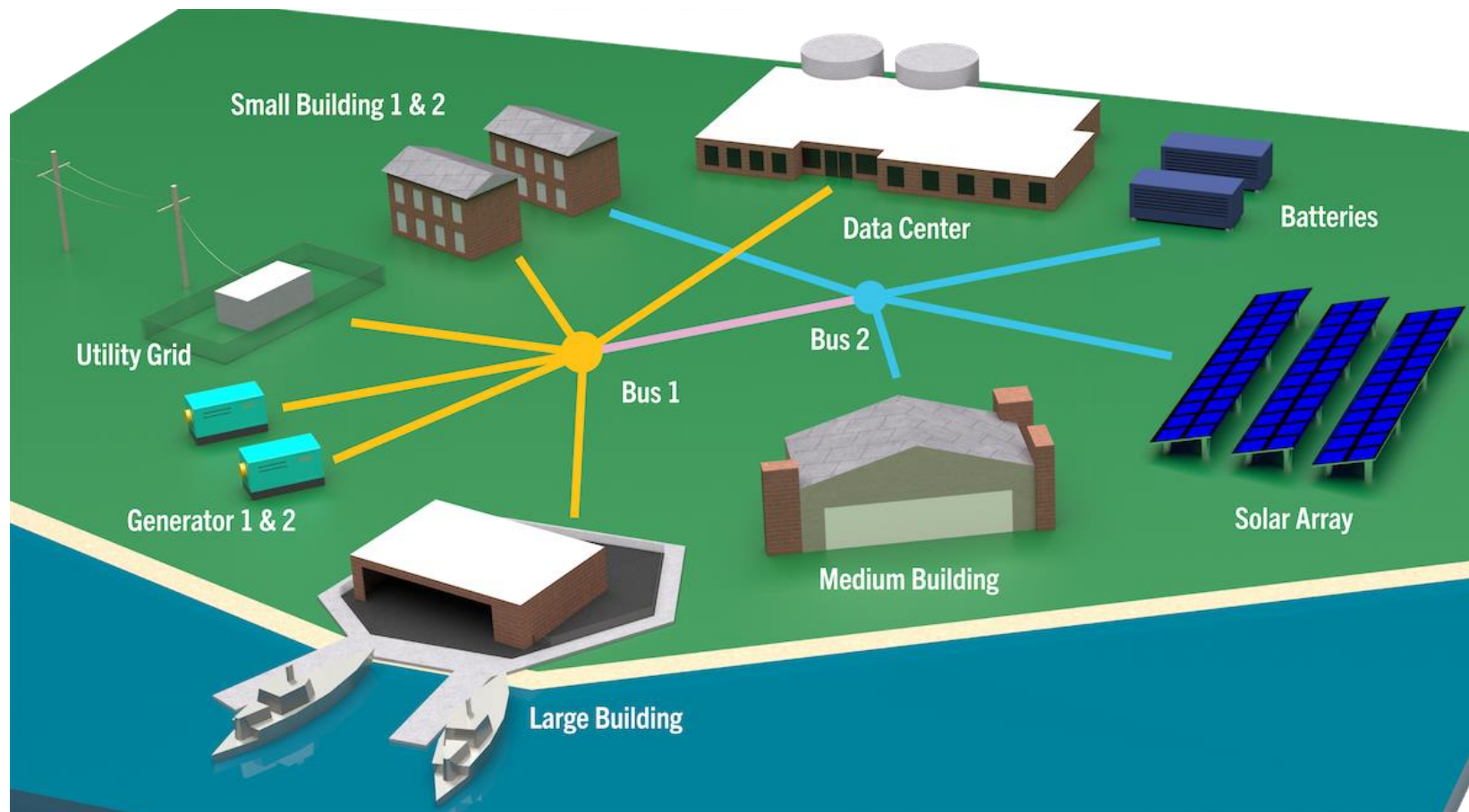
DTs are systems synchronized with another system of interest



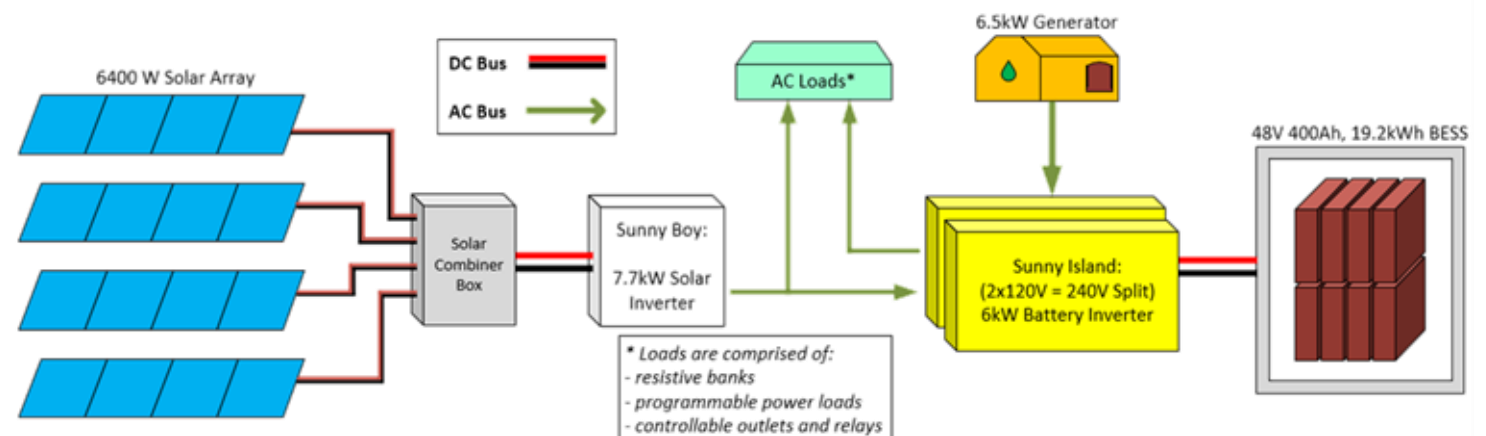
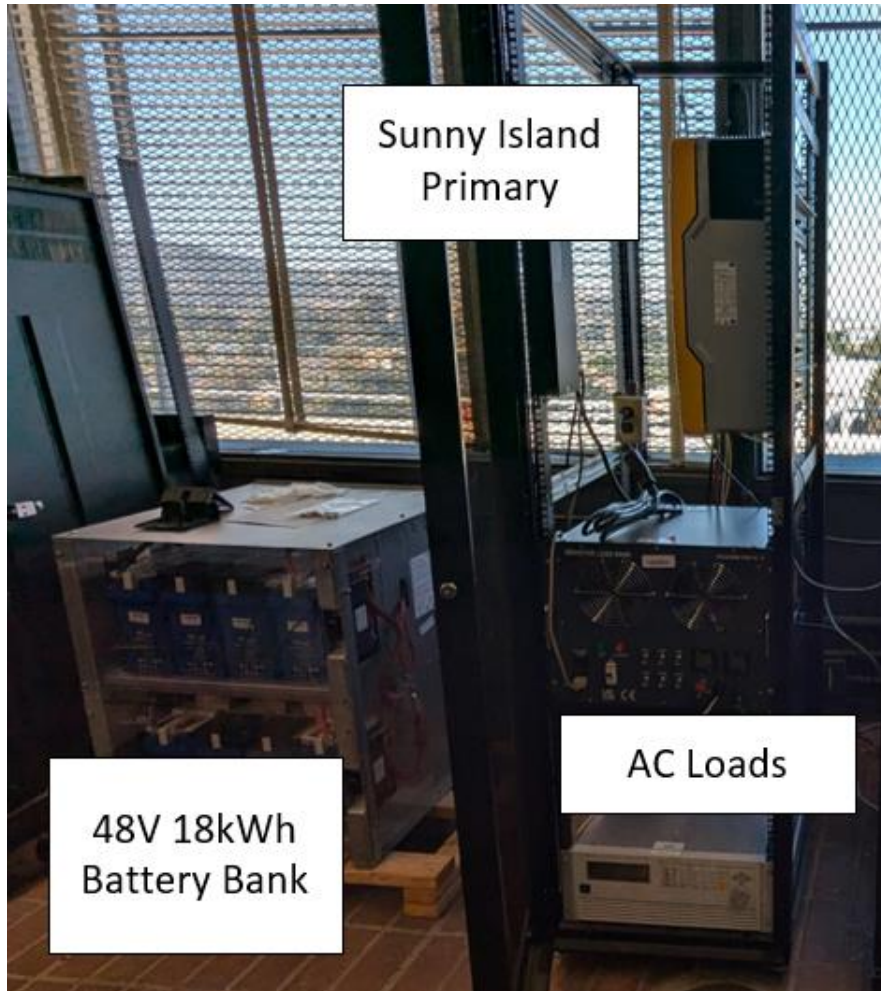
Digital Twins are 2 systems sharing the same CHG



Demonstration: Microgrid Digital Twin



Physical Microgrid at Naval Postgraduate School

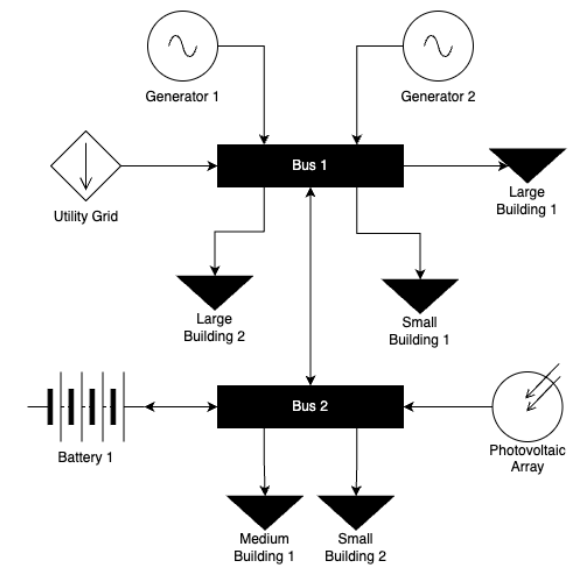
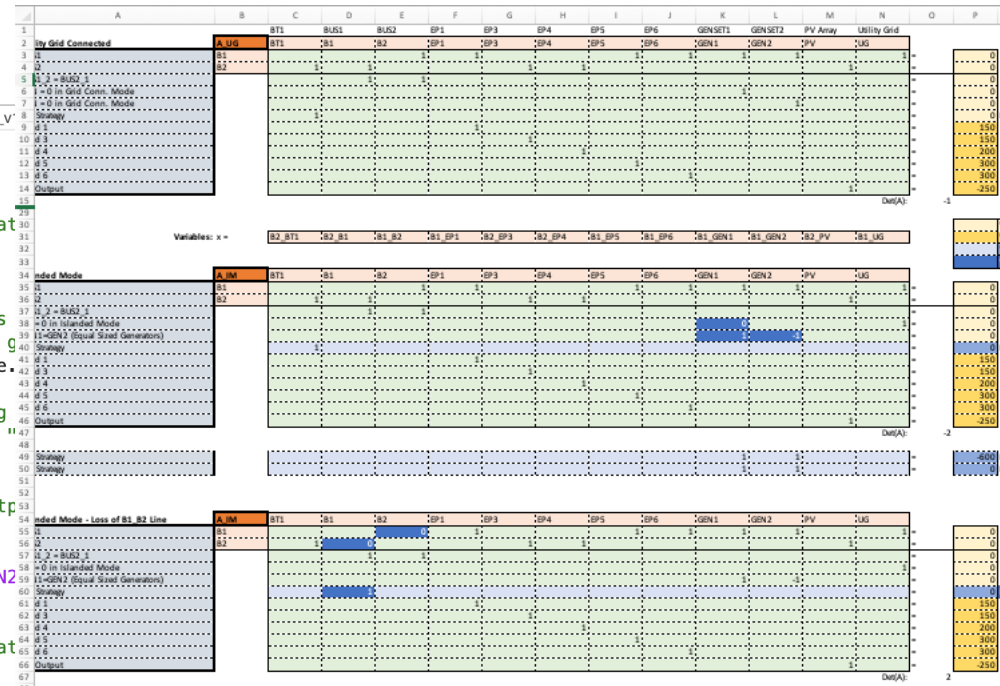


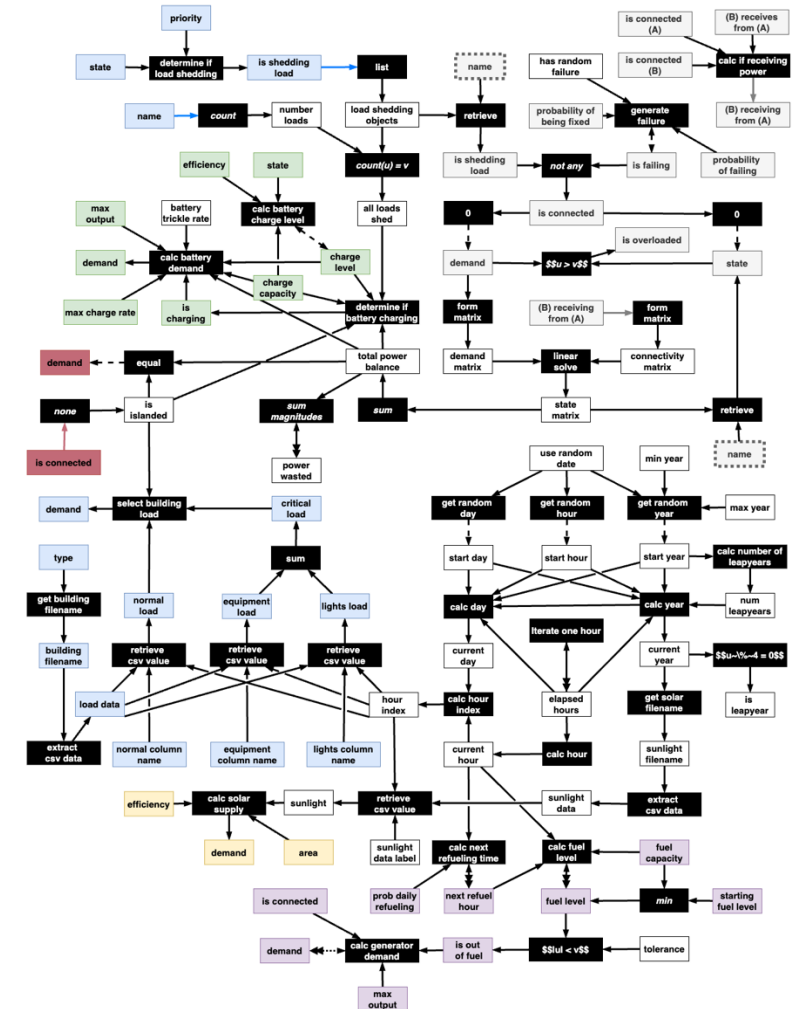
Previous Imperative Modeling of Microgrid

```

MG_LP_Test_v10.m ×
/Users/john_morris/Documents/Clemson/Research/MicrogridHg/archived_model/MG_LP_Test_v
423 %Initial Solve
424 x(:,n) = linsolve(A2,b);
425
426 %Determine if generator demand exceeds generat
427 Gen_Demand = -x(pos.Gens,n);
428
429 if any(Gen_Demand > [Gens.Capacity])'
430     %If the B2 to B1 Buss line or BT1 line is
431     %exhausted cannot utilize ESS to make up g
432     if MG_State.B2_B1(n) == false || MG_State.
433         || BT1_Charge(n) < 0
434         % Below line was useful for debugging
435         % disp("Gens Overloaded at time step "
436         Overload.Gens = true;
437     else
438         %Otherwise set Generators at full outp
439         %make up unmet demand
440         A2(6,:) = pos.Gens & MG_State{n,:};
441         b(6) = -MG_State{n,{'B1_GEN1' 'B1_GEN2
442         x(:,n) = linsolve(A2,b);
443     end
444     %If Generator demand is negative, then generat
445     %charge batteries with excess PV generation
446     elseif any(Gen_Demand < 0)
447         A2(6,:) = pos.Gens;
448         b(6) = 0;
449         x(:,n) = linsolve(A2,b);
450     end
451
452 % Check for Battery Output Exceeded
453 % Add 0.1 Due to Rounding Errors in Linear Solver
454 if -x(contains(LoadVars,'BT1'),n) - 0.01 > BT1.Output * (BT1_Charge(n) > 0)
455     Overload.BT1 = true;
456     % Below line was useful for debugging in single runs. Commented out

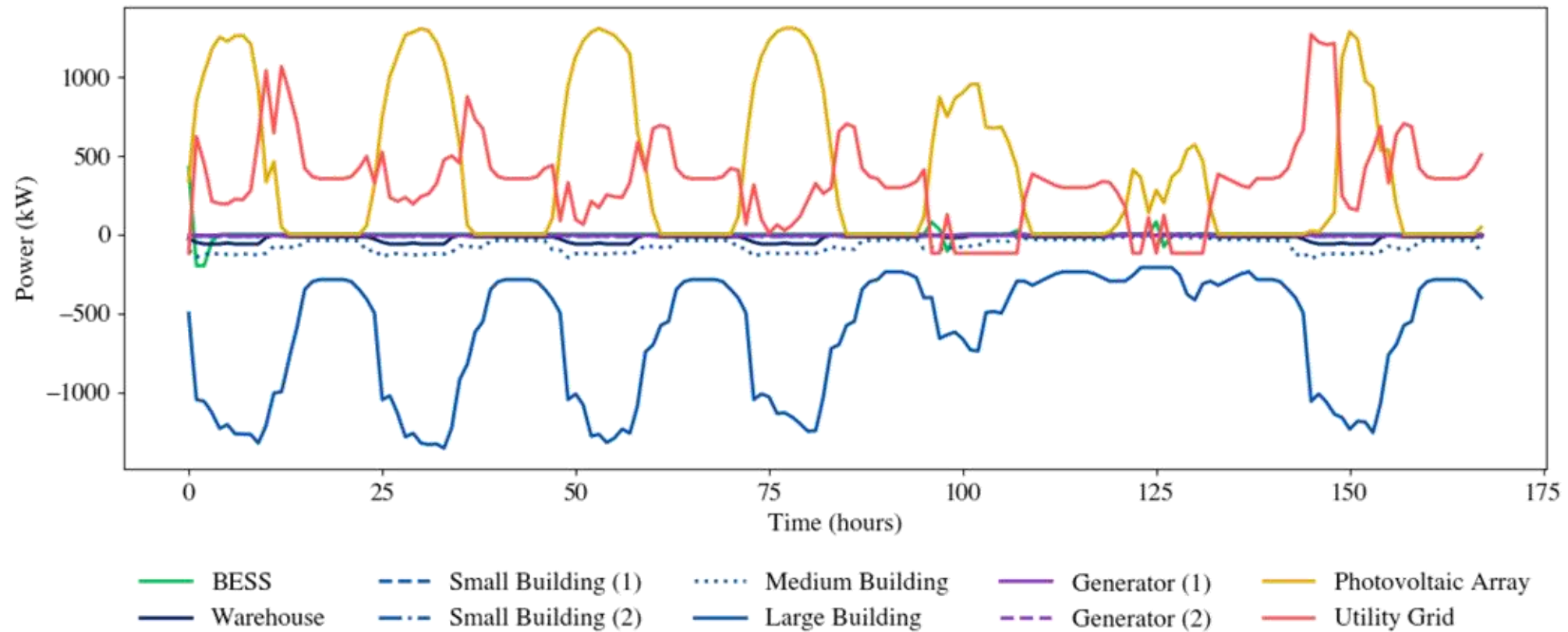
```



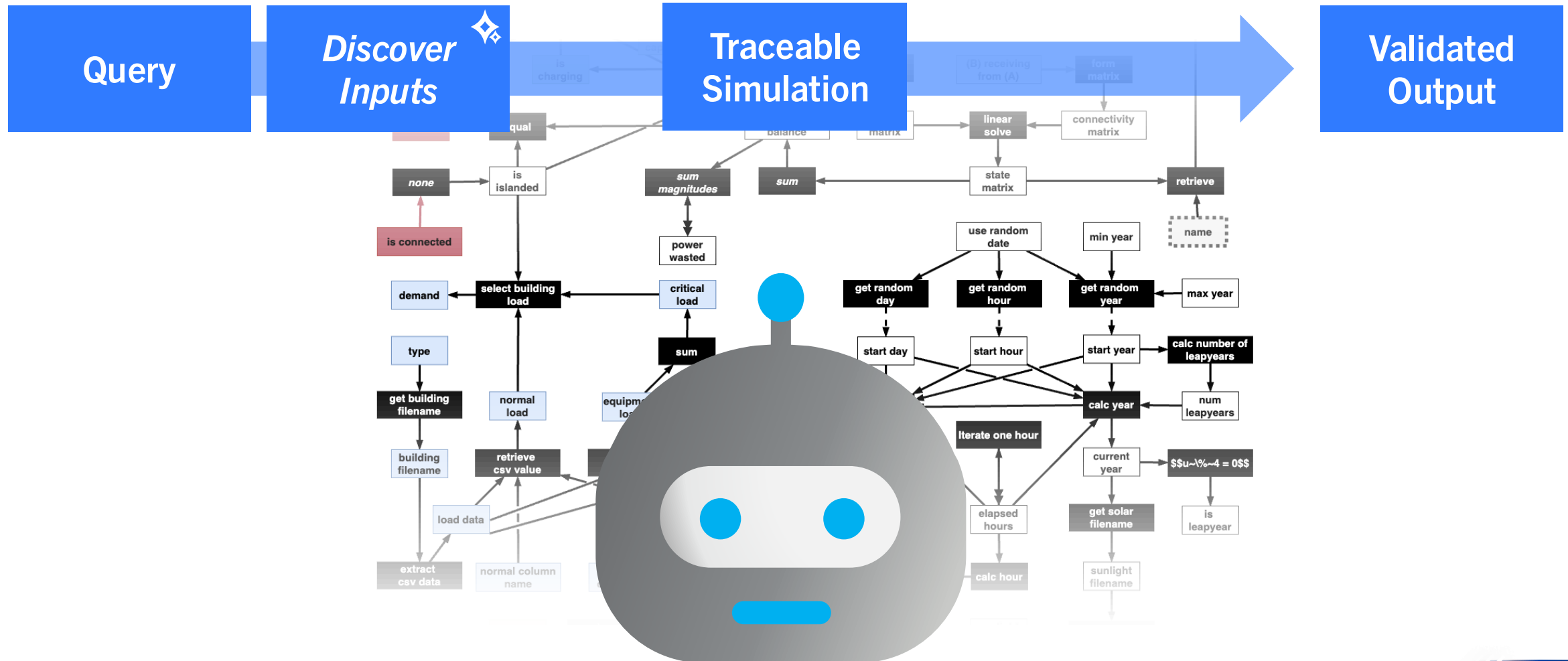




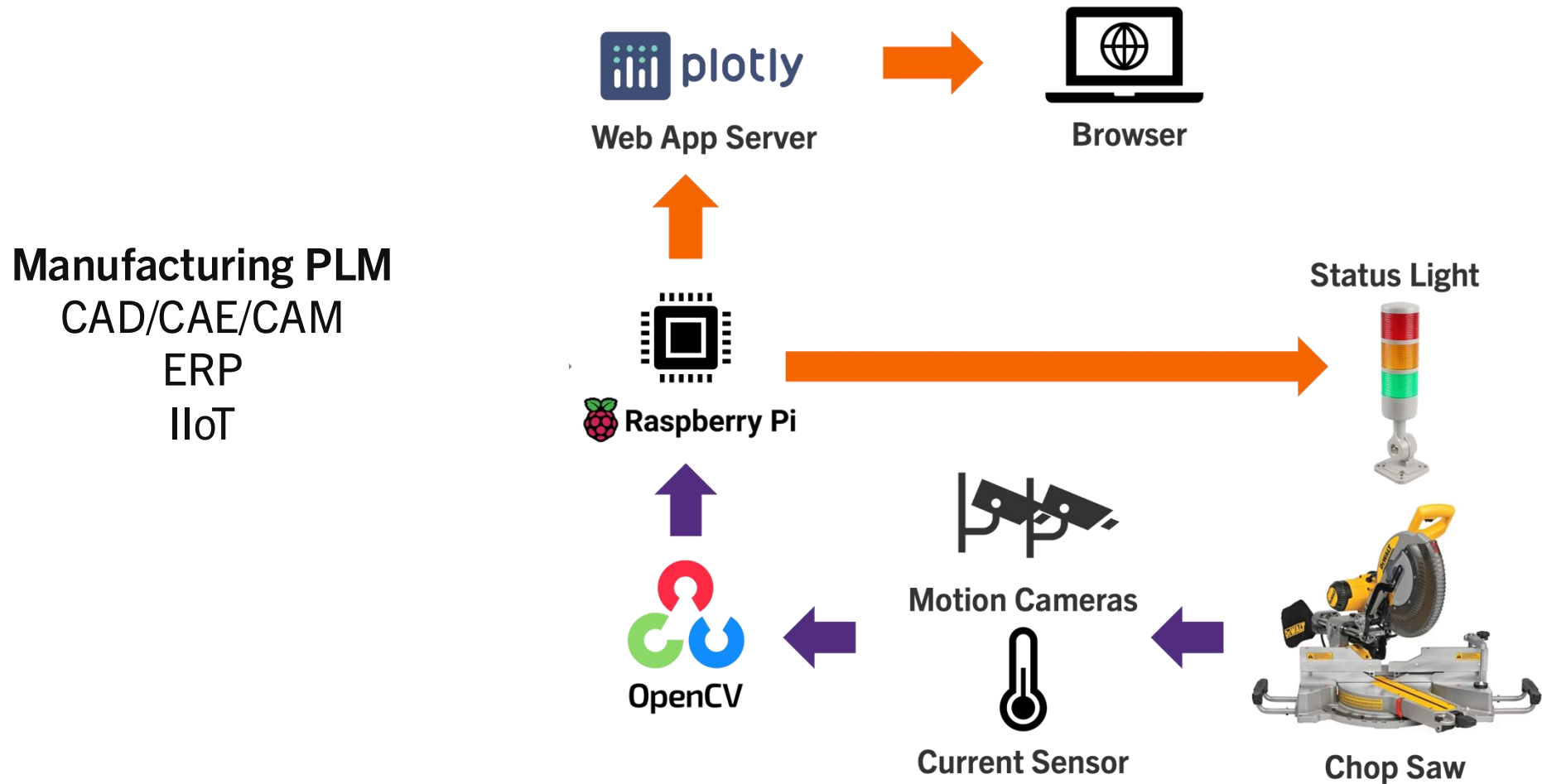
Autonomously observes and simulates the real system



Reasoning is provided by automatic path composition, leading to deterministic, explainable AI

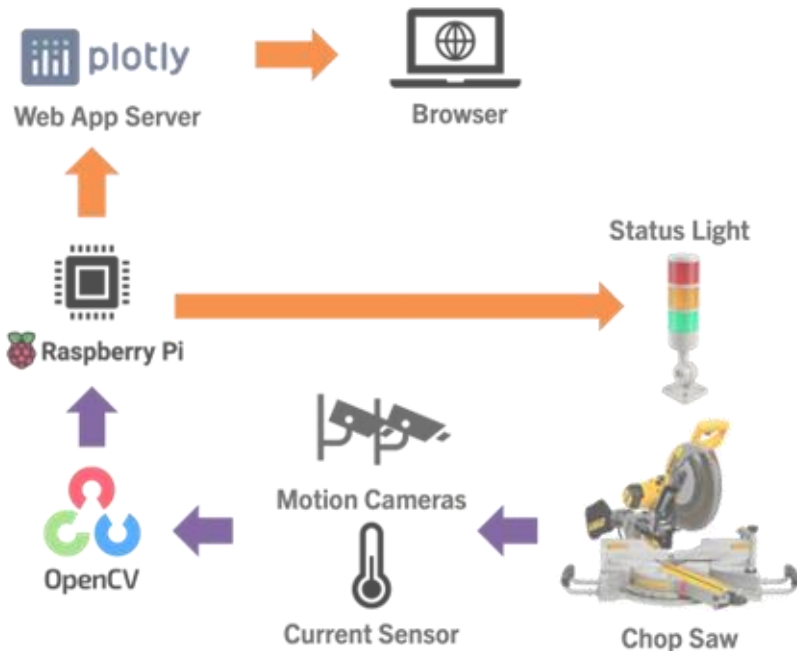


Needed for domains with complex information integration

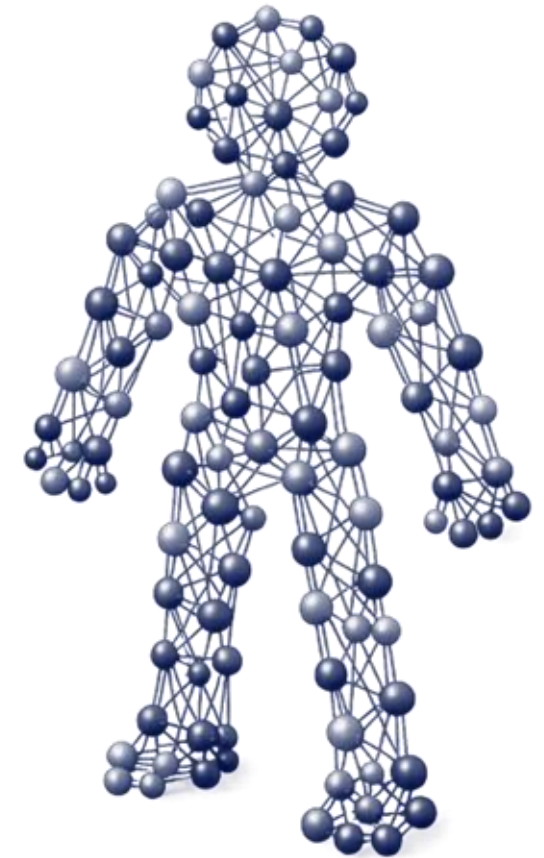


Needed for domains with complex information integration

Manufacturing PLM
CAD/CAE/CAM
ERP
IIoT

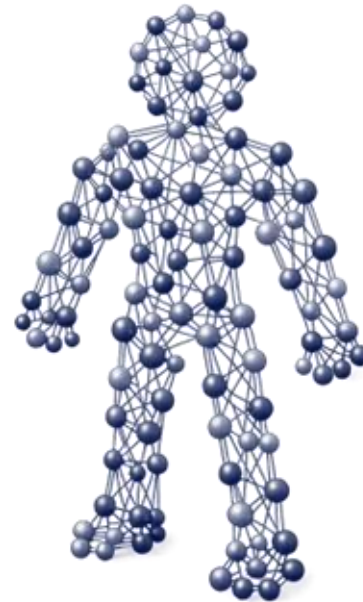
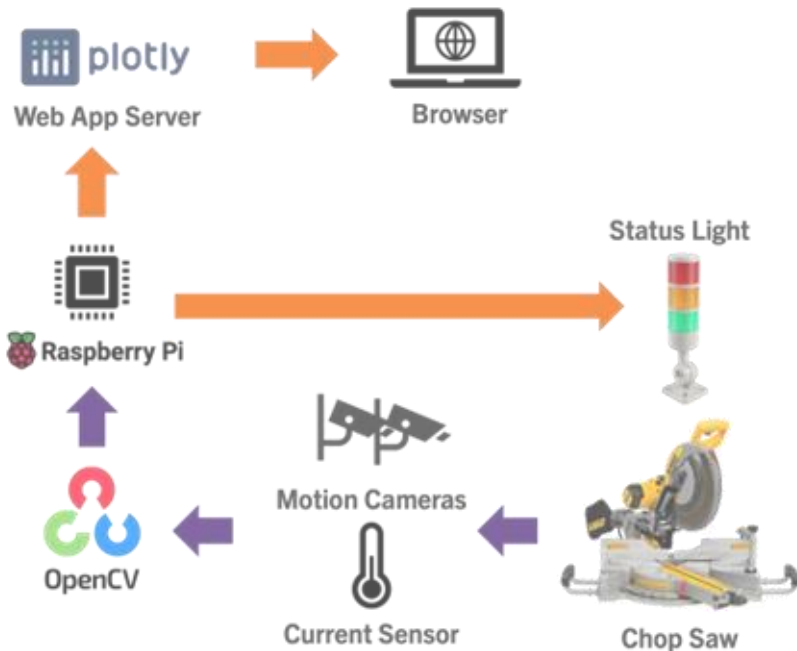


Personal Digital Twins
Medical records
Monitored biometrics
Therapeutic diagnostics



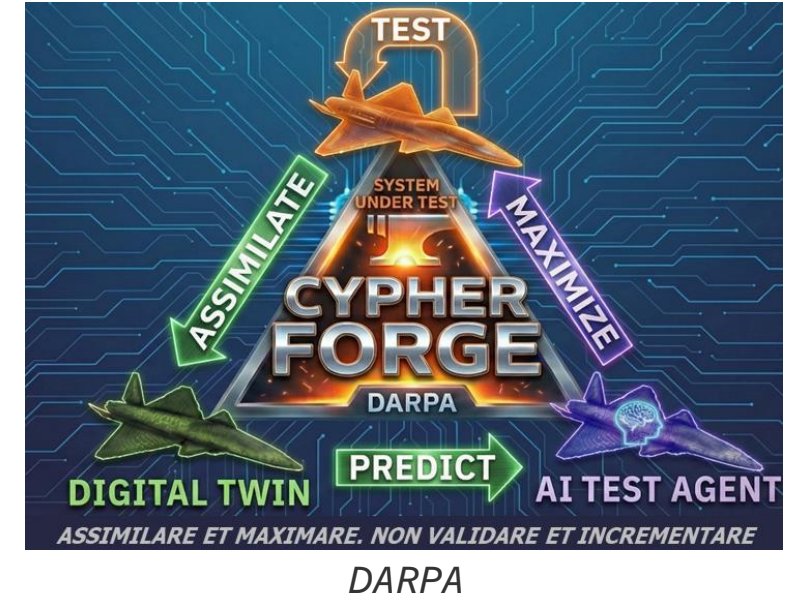
Needed for domains with complex information integration

Manufacturing PLM
CAD/CAE/CAM
ERP
IIoT



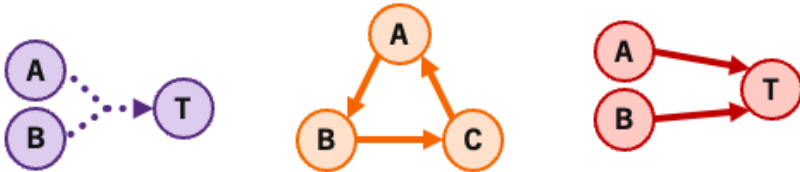
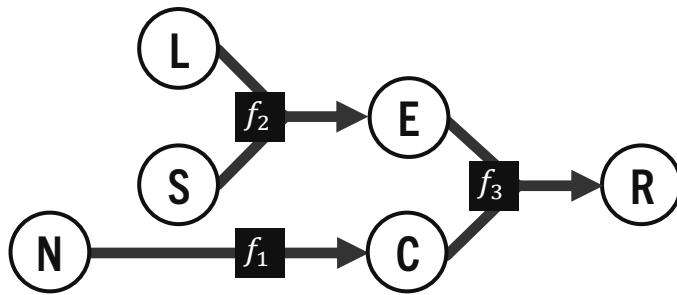
Personal Digital Twins
Medical records
Monitored biometrics
Therapeutic diagnostics

System Planning
Procurement
Design
Validation and Verification



The structure of a CHG allows it to represent any system

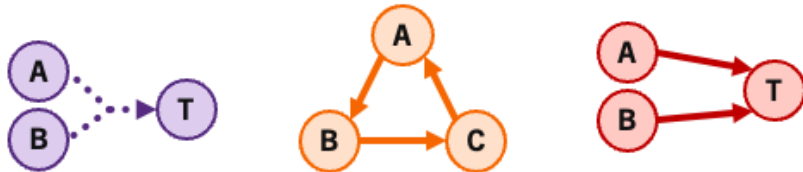
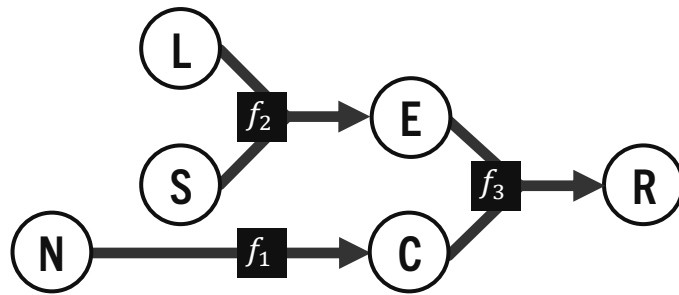
Universal Modeling



Information about a system can be discovered autonomously

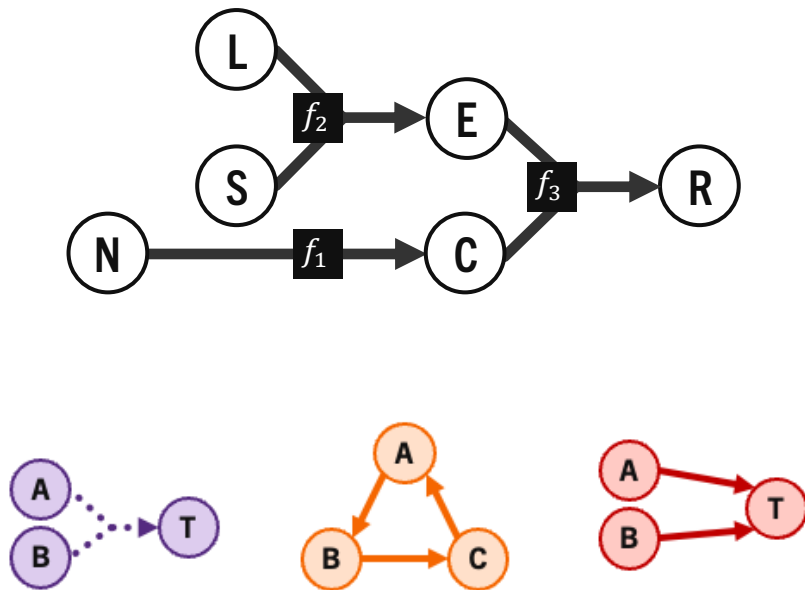
Universal Modeling

Declarative Simulation



Models are connected across domains, scales, and tools

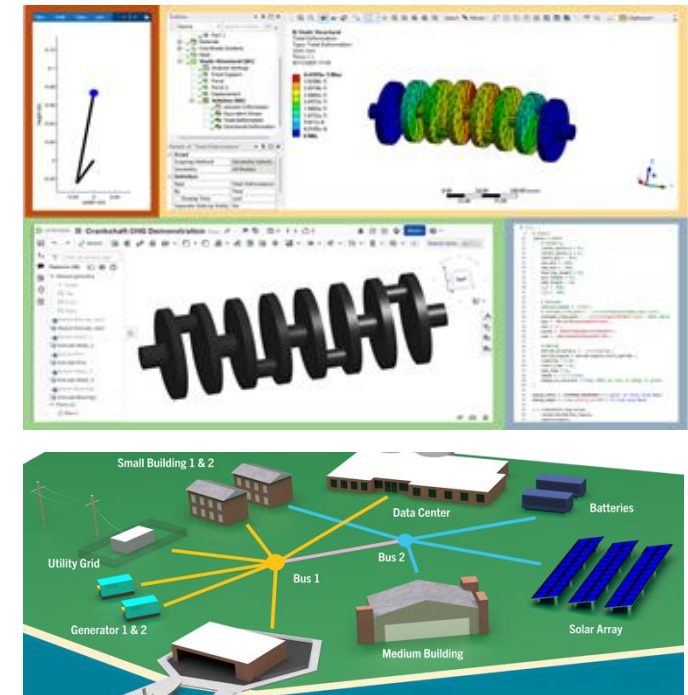
Universal Modeling



Declarative Simulation

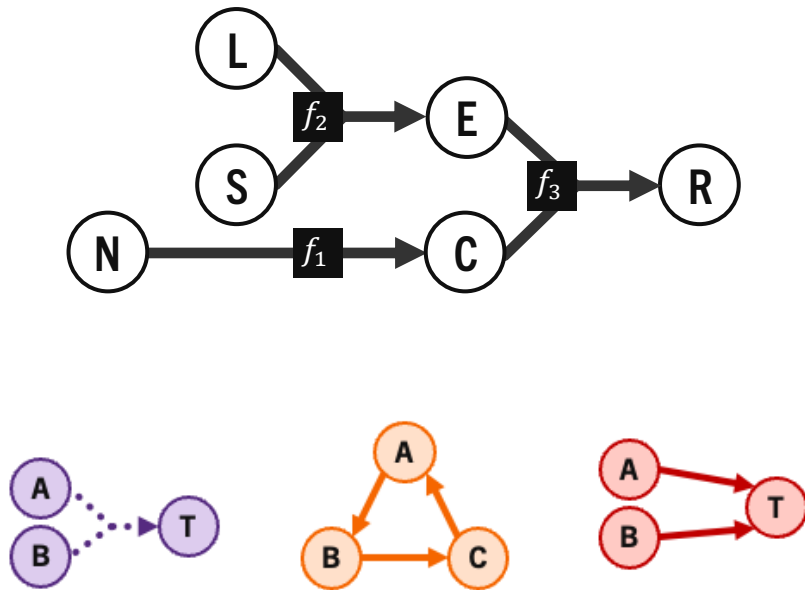


Executable Digital Threads



CHGs provide a universal language for declarative systems modeling

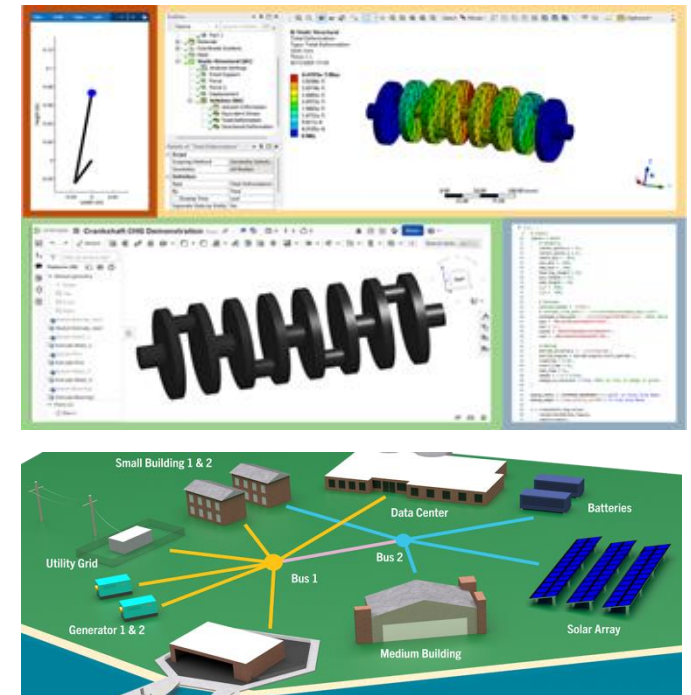
Universal Modeling



Declarative Simulation



Executable Digital Threads



John Morris
jhmrrs@clemson.edu